Finite-Dimensional Type I von Neumann Algebras in PyTorch: A GPU-Accelerated Framework for Random Block-Diagonal Operators

Irina Nikolaeva Andrej Novikov

Abstract

Finite-dimensional von Neumann algebras, the direct sums $\bigoplus_c M_{k_c}(\mathbb{C})$ of matrix algebras, are basic objects of operator algebra theory and, through the commutant of a symmetry, the natural home of quantum systems with conserved charges or superselection rules. To our knowledge no numerical software treats them as a data type, with their centre, their family of traces, conditional expectations, inclusions and functional calculus. `torch_vn_algebra` is a PyTorch library that does, for batches of operators on CPUs and GPUs and with automatic differentiation. It began as a re-implementation of our earlier NumPy Monte Carlo study of trace inequalities with the Michelson contrast, and this paper documents it. Its design rests on one thesis: keeping the sector structure inside the data type makes computations in the algebra correct by construction and cheap. Haar sampling is exact in every block, trace conventions survive maps between sectors, and all blocks of one size go through one batched kernel. We show that the library reproduces the distributions of the earlier NumPy computation and is about two orders of magnitude faster than its loop formulation for small blocks. More important than speed, every operation is differentiable, which turns the sampling of the earlier study into optimisation: from block size 4 on, a million Haar samples miss the violation of a trace inequality in every one of 16 test cases, while batched gradient ascent finds the exact supremum in all of them within seconds. We measure each architectural decision against the simple alternative and validate the library against exact results. Its generality is illustrated by symmetry verification for a noisy XY mixer of the quantum alternating operator ansatz, whose Hamming-weight sectors form the algebra. One batched run gives, for all noise models and rates, the exact acceptance probabilities and post-selected fidelities. It also gives the first-order dephasing integral and a gain from post-selection that grows with the register size.

Program summary

Program title: `torch_vn_algebra`

Developer's repository link: `https://github.com/andre-a-no/von_neumann_type_i`

Licensing provisions: MIT

Programming language: Python (≥ 3.8), PyTorch (≥ 1.13), NumPy (≥ 1.21)

Nature of problem: Numerical experiments in finite-dimensional Type I von Neumann algebras $\bigoplus_c M_{k_c}(\mathbb{C})$ and in the quantum systems with superselection rules or conserved charges that they describe: random operators with prescribed spectra, several trace functionals, states, channels within and between sectors, symmetric tensor products, spin chains, Hamiltonian and Lindblad dynamics with jumps between sectors (noisy circuits with a conserved charge), extremal problems under spectral constraints, inclusions of algebras, symmetry reduction, and quantum Rényi divergences.

Solution method: Batched block-diagonal tensors processed by PyTorch linear algebra on CPUs and GPUs, one call per block size; lazy operators that carry their properties, with caches for spectra; exact Haar sampling; spectral functional calculus with Daleckii–Krein derivatives; Kraus representations; Lanczos and Krylov methods for sparse sector Hamiltonians; sparse jumps between sectors; batched multistart gradient optimisation; run-time and memory estimates before expensive steps; inclusion matrices; randomised Wedderburn decomposition; highest-weight bases and exact Clebsch–Gordan coefficients.

Additional comments: more than 800 unit tests, run in continuous integration; every number, table and figure of this paper is produced by the scripts and examples in the repository and regenerated by `scripts/run_all.sh`. The library is the software used in [9] and in the companion papers [1, 2, 3].

1 Introduction

The algebra as the object. A finite-dimensional von Neumann algebra is, up to $*$-isomorphism, a direct sum of full matrix algebras [25],

$$\mathcal{M} = \bigoplus_{c=1}^{C} M_{k_c}(\mathbb{C}) \quad \text{acting on} \quad \mathcal{H} = \bigoplus_{c=1}^{C} \mathbb{C}^{k_c}, \tag{1}$$

where $M_k(\mathbb{C})$ is the algebra of complex $k \times k$ matrices and the C summands are called sectors. Small as they are, these algebras carry the structure that operator algebra theory studies. Their centre is spanned by the sector projections. Instead of one trace they have a C-parameter family of faithful traces. They have conditional expectations onto subalgebras, inclusions with an index [63, 64], and a Jordan product on their self-adjoint part; their elements have spectral invariants such as the Michelson contrast, a normalised spread of the spectrum (Section 3). Questions about these objects are often explored on examples first: which inequalities between traces of products characterise the trace or the centre [5, 6, 7, 8], and how strongly they fail for functionals that are not traces [9]. Such explorations are usually written as ad hoc matrix code. To our knowledge there is no numerical software in which the algebra (1) is the data type: one declares the sector sizes and then samples, transforms and optimises elements, states and maps, with the centre and the traces built in (Section 2).

From an earlier NumPy study to a library. This library grew out of such an exploration. In [9] and in the first version of this work [4] we studied trace inequalities with the Michelson contrast by Monte Carlo sampling in NumPy. The computation draws many positive operators with prescribed contrast and Haar-random eigenvectors in every block, evaluates a trace functional, and looks at its distribution. Written directly, such code loops over samples and sectors, restates the trace convention in every script, and has to get the Haar measure right block by block.[1] `torch_vn_algebra` reimplements that modelling as a structured PyTorch library. Section 4 shows that it reproduces the earlier computation: the NumPy formulations and the library sample the same distribution. For small blocks the library is about two orders of magnitude faster than the loop formulation, and for blocks of size 16 about three times faster, and the same code runs on a GPU.

From sampling to optimisation. Speed is the smaller gain. Every operation of the library is differentiable, including spectral functions (by the Daleckii–Krein formula), channels between sectors and the time steps of Hamiltonian and Lindblad dynamics, so a question asked by sampling can

[1] The first version [4] contained errors of this kind in its samplers; they are corrected and tested here.

be asked again as a variational problem: maximise a functional over operators with a prescribed spectrum, over states, over unitaries of the algebra or over all channels, by batched gradient ascent with many starts in one call. This changes the answers, not only the time. The Monte Carlo of [9, 4] saw no violation of Gardner's inequality for blocks of size 16, and we now know why: by concentration of measure, the unitaries near the maximiser have Haar measure that decays rapidly with the dimension. From block size 4 on, a million Haar samples miss the violation in every test case, while gradient ascent finds the exact supremum in all of them, to a relative error of 3×10^{-5} and in a few seconds (Section 8). The violation envelopes of the companion paper [1], the thresholds of the Rényi divergences in Section 8.1 and the partitions in [2] are all found this way; every reported extremal value is then recomputed at the optimiser in 50-digit arithmetic, so it is a certified lower bound for the supremum.

Thesis. The design rests on one thesis: keeping the sector structure inside the data type makes computations in (1) both correct by construction and cheap. By sector structure we mean the blocks, the centre, the several traces and the maps between algebras with different sectors. Correctness follows because a random element is sampled block by block from the right measure, a density is always stored with respect to one trace and converted explicitly, and a map between sectors carries its own dual. Cheapness follows because the library knows which blocks have equal size and stacks them into one batched kernel call, knows from property flags which kernel a result needs, and never forms the zero blocks. Section 5 describes the architecture built on this thesis and measures each decision against the simple alternative.

Where the algebra appears in physics. The same algebra is the home of quantum systems with a symmetry. If a group, or a set of conserved quantities, acts unitarily on $\mathcal{H} \cong \bigoplus_\lambda V_\lambda \otimes \mathbb{C}^{m_\lambda}$, with irreducible representations V_λ of multiplicity m_λ, the operators commuting with the action form the algebra $\bigoplus_\lambda \mathbb{1}_{V_\lambda} \otimes M_{m_\lambda}(\mathbb{C}) \cong \bigoplus_\lambda M_{m_\lambda}(\mathbb{C})$. For one conserved charge Q (particle number, magnetisation, Hamming weight) the sectors are the eigenvalues of Q and the centre consists of the functions of Q; a Hamiltonian that conserves Q, the evolution it generates and every observable compatible with a superselection rule for Q lie in this algebra. Examples are superselection rules [26, 27] and decoherence [28], many-body Hamiltonians diagonalised sector by sector [29, 30], and symmetry-resolved entanglement [31, 32]. Others are decoherence-free subspaces and operator-algebra error correction [33, 34, 35], the XY mixers of the quantum alternating operator ansatz [38, 105], random matrices decomposed by symmetry [39, 15], and equivariant layers in machine learning, which lie in such commutants by Schur's lemma [40, 41, 72]. A library built for trace inequalities therefore serves these applications once it has dynamics and maps that move probability between sectors. We added them (spin chains with $U(1)$ and SU(2) sectors, Lindblad dynamics with jumps between sectors, inclusions, a numerical Wedderburn decomposition), and Section 11 uses them for a question that the original NumPy code could not address: symmetry verification in a noisy XY mixer.

Why not one large matrix? $\mathcal{M}$ embeds into $M_D(\mathbb{C})$, $D = \sum_c k_c$, as the block-diagonal matrices, and every matrix package handles that embedding exactly. The question is what it costs and what has to be supplied by hand.

- *Cost.* A product or a decomposition costs $O(D^3)$ time and $O(D^2)$ memory in M_D, but $\sum_c k_c^3$ and $\sum_c k_c^2$ in $\mathcal{M}$: for C equal blocks a factor C^2 in time and C in memory.
- *Measures.* The unitary group of $\mathcal{M}$ is $\prod_c U(k_c)$, for $C \geq 2$ a null set in $U(D)$. A Haar-random element of $U(D)$ is not in $\mathcal{M}$; random elements, random states and gradient steps have to respect the blocks.

- *Traces.* The faithful traces of $\mathcal{M}$ form the family $\sum_c w_c \operatorname{Tr}_c$. Densities, "trace preserving" and the dual of a map between algebras with different sectors depend on the weights, which the embedding forgets.
- *Centre.* Conditioning on a sector, post-selection and sector-resolved quantities are operations with the centre; in M_D they have to be reconstructed from block indices.

For $C = 1$ the library reduces to batched linear algebra in $M_k(\mathbb{C})$, and all its functions accept that case.

What is new and what is re-implemented. None of the numerical methods is new. Haar unitaries come from the QR decomposition of Gaussian matrices with Mezzadri's phase correction [14]. Gradients of matrix functions use the Daleckii–Krein formula [50, 51]. Singular values come from the Hermitian dilation, holomorphic functions from contour integrals [49], and the Wedderburn decomposition from the randomised algorithms of Murota and coauthors [69, 71]. Block-sparse storage of charge-conserving tensors is standard in tensor-network libraries [45, 47, 46]. Lazy linear operators that dispatch on their structure are the basis of GPyTorch's `linear_operator` [89, 90] and of CoLA [91]. Our contribution is their combination around the algebra (1) and its conventions: a batched, differentiable data type whose elements know their sector structure, trace convention and spectral properties, with states, channels between algebras, inclusions and dynamics defined on it.

What the paper shows, and for whom. Section 4 reproduces the earlier NumPy study and compares speed. Section 5 explains where the speed comes from, with one measurement per design decision (Table 5), and Table 3 lists seven implementation problems and the sections that resolve them. Monte Carlo estimates agree with exact Haar, Weingarten, Page and purity values within their standard errors (Section 7), and batched optimisation finds extremes that sampling misses (Section 8). Sections 9 and 10 add spin chains, inclusions and symmetry reduction, and Section 11 shows what this generality buys. The intended users are operator algebraists who test conjectures numerically, quantum information researchers who work with superselection rules or symmetric noise, and physicists who need many small symmetric problems, such as disorder realisations or noise sweeps, in one batch. The library is not a circuit simulator or a tensor-network code; for single large systems the packages of Section 2 remain the better choice. This paper describes the software used in [9] and in three companion papers [1, 2, 3]; their results are summarised in Section 12 and not repeated.

2 Related software

Software that touches the algebra (1) falls into groups that store matrices or sector blocks for a purpose, usually physics, rather than treating the algebra itself as the object. Table 1 summarises the features relevant here.

Linear algebra. NumPy and SciPy [42, 43], with ARPACK [44] for sparse eigenproblems, handle dense and sparse matrices; stacked arrays give batching, and any block structure is maintained by hand. GPyTorch's `linear_operator` [89, 90] and CoLA [91] go further. They represent structured matrices (diagonal, Kronecker, block-diagonal, low-rank, sums and products) lazily and choose solvers and decompositions by structure, on GPUs and with automatic differentiation. The lazy operators of Section 5.3 follow the same idea. What these packages do not provide is the operator-algebraic layer: the centre, several traces, states, and maps between algebras.

Quantum dynamics. QuTiP [10, 11] and its Julia counterpart QuantumToolbox.jl [80] represent states, operators and superoperators on one Hilbert space and solve Schrödinger, Lindblad and stochastic equations. QuantumToolbox.jl runs on GPUs and supports automatic differentiation; QuTiP 5 does so through optional JAX and CuPy data backends. QuTiP includes PIQS [79], which uses permutational invariance to block-diagonalise the Lindblad dynamics of N two-level systems. PIQS is the closest existing analogue of the sector-wise computations here, specialised to one symmetry. `dynamiqs` [13] provides batched, differentiable Schrödinger and Lindblad solvers on GPUs, built on JAX. Circuit simulators propagate density matrices under noise: Qiskit Aer [12, 81] with noise models on CPUs and GPUs, QuEST [82] on CPUs, GPUs and clusters, and NVIDIA's cuQuantum SDK [83], whose cuDensityMat library [84] integrates master equations for batched operators on GPUs. PennyLane [85] differentiates hybrid quantum-classical circuits, including mixed-state simulation. All of these work on one Hilbert space; a conserved charge, if it is used, is exploited by the user.

Symmetry sectors. Exact-diagonalisation packages build symmetry-reduced bases and sparse sector Hamiltonians: QuSpin [30, 86] for spins, bosons and fermions, and XDiag [87], written in C++ with a Julia interface and distributed over many cores. Tensor-network libraries store charge-conserving tensors block-sparsely: ITensor [45], TeNPy [47], TensorKit.jl [88], and YASTN [46], which, like this library, runs on PyTorch. These packages solve large single problems much better than ours. They do not offer batches of random elements, several traces or maps between algebras.

Algebra software. To our knowledge, no existing numerical library treats finite-dimensional von Neumann algebras $\bigoplus_c M_{k_c}$ as a data type with a centre, traces and states, conditional expectations, inclusions with their index, and functional calculus. Computer-algebra systems provide exact structure theory for small algebras. Examples are Euclidean Jordan algebras with spectral and Peirce decompositions in Orlitzky's SageMath package [92], the Jordan algebras of SageMath, the GAP package ALCO [93], an R package [94], and centres, radicals and algebras with involution in SageMath, GAP and Magma [95]. Numerical work block-diagonalises matrix $*$-algebras as a preprocessing step for semidefinite programming [69, 70, 96, 97]. In the same vein, RepLAB [98] decomposes representations of compact groups by sampling their commutant, and SymbolicWedderburn.jl [99] computes Wedderburn decompositions of group-symmetric problems exactly. RepLAB's commutant object is the closest numerical analogue; it has no centre or state interface, inclusions, index or functional calculus. Conic solvers hard-code the Jordan products of specific symmetric cones [100, 101]. Subfactor software is combinatorial, working with principal graphs and planar algebras [102, 103]. Proof assistants formalise von Neumann algebras and continuous functional calculus without computing with them [104]. `torch_vn_algebra` is numerical, batched, GPU-capable and differentiable, which is what Monte Carlo sampling and optimisation over the algebra need; it takes the block structure as input, and its own Wedderburn decomposition (Section 10) can also supply it.

3 Mathematical setting

Algebra and traces. Let $\mathcal{M} = \bigoplus_{c=1}^{C} M_{k_c}$ act on $\mathcal{H} = \bigoplus_c \mathbb{C}^{k_c}$; an element $A \in \mathcal{M}$ is block diagonal, $A = \bigoplus_c A_c$. Every trace on $\mathcal{M}$ is $\mathrm{tr}_w(A) = \sum_c w_c \operatorname{Tr} A_c$ with weights $w_c \geq 0$, faithful iff all $w_c > 0$. The library implements three of them,

$$\mathrm{Tr}_{\mathrm{blunt}} : \ w_c = 1, \qquad \mathrm{Tr}_{\mathrm{norm}} : \ w_c = \tfrac{1}{k_c}, \qquad \tau_{\mathrm{vN}} : \ w_c = \tfrac{1}{C k_c}.$$

$\mathrm{Tr}_{\mathrm{blunt}}$ is the plain trace of the block-diagonal matrix; we call it "blunt" because it ignores the block sizes. $\mathrm{Tr}_{\mathrm{norm}}$ normalises each block, and τ_{vN} is the faithful tracial state that gives every sector

	NumPy/SciPy	QuTiP 5 (PIQS)	QuantumToolbox.jl	dynamiqs	Qiskit Aer, QuEST	cuDensityMat	QuSpin, XDiag	ITensor, TeNPy, YASTN	lin. operator, CoLA	this work
GPU execution	–	(✓)	✓	✓	✓	✓	–	(✓)	✓	✓
batched over many problems	(✓)	–	(✓)	✓	–	✓	–	–	✓	✓
automatic differentiation	–	(✓)	✓	✓	–	(✓)	–	(✓)	✓	✓
noisy (Lindblad) dynamics	–	✓	✓	✓	✓	✓	–	–	–	✓
symmetry sectors	–	(✓)	–	–	–	–	✓	✓	(✓)	✓
centre, several traces, cond. expectations	–	–	–	–	–	–	–	–	–	✓
maps between algebras of different sectors	–	(✓)	–	–	–	–	–	–	–	✓
Haar/COE/CSE and prescribed spectra per sector	(✓)	(✓)	–	–	–	–	–	–	–	✓
large single systems	✓	✓	✓	✓	✓	✓	✓	✓	✓	(✓)

Table 1: Features relevant to computations in the algebra (1), to the best of our knowledge in October 2026 (✓: supported; (✓): partly, through optional backends or plug-ins, for one symmetry (PIQS), or with user code; –: not supported). QuTiP 5 uses GPUs and automatic differentiation through its JAX and CuPy backends; PIQS handles the sectors of permutational symmetry and the maps between them. Qiskit Aer runs density-matrix simulations with noise models on GPUs, QuEST on GPUs and clusters. The last row is where the specialised packages are the better choice.

the same weight. Empty sectors ($k_c = 0$) are allowed, carry no weight, and are not counted in C for τ_{vN}. The library also accepts nominal factor sizes $n_c \geq k_c$ and then works in the corner $p\mathcal{M}_0 p \cong \bigoplus_c M_{k_c}$ of $\mathcal{M}_0 = \bigoplus_c M_{n_c}$ (p_c of rank k_c); all traces refer to the corner, and in most applications $k_c = n_c$. For $A \in \mathcal{M}$, $|A| = (A^*A)^{1/2}$. For $A \geq 0$, $A \neq 0$, the *Michelson contrast* is $\Delta(A) = (\lambda_{\max} - \lambda_{\min})/(\lambda_{\max} + \lambda_{\min}) \in [0, 1]$, with the extreme eigenvalues taken over all sectors; it is the operator version of the fringe visibility of optics, and $\Delta(A) = 0$ iff A is a multiple of the identity (the library returns $\Delta(0) = 0$).

States. A state ω on $\mathcal{M}$ is represented by its density $\rho \geq 0$ with respect to $\mathrm{Tr}_{\mathrm{blunt}}$, $\omega(A) = \mathrm{Tr}(\rho A)$, $\sum_c \mathrm{Tr}\,\rho_c = 1$. Its restriction to the centre is the classical distribution $p_c = \mathrm{Tr}\,\rho_c$ over sectors. With respect to tr_w the same state has the density $\rho_c^{(w)} = \rho_c / w_c$; the library converts between conventions explicitly.

Maps between sectors. Let $\mathcal{M}_{\mathrm{in}} = \bigoplus_c M_{k_c}$ and $\mathcal{M}_{\mathrm{out}} = \bigoplus_d M_{m_d}$. Every normal completely positive map $\Phi : \mathcal{M}_{\mathrm{in}} \to \mathcal{M}_{\mathrm{out}}$ (Schrödinger picture) is a sum of completely positive maps between blocks,

$$\Phi(\rho)_d = \sum_c \sum_i K_i^{dc}\, \rho_c\, (K_i^{dc})^*, \qquad K_i^{dc} : \mathbb{C}^{k_c} \to \mathbb{C}^{m_d}.$$

It preserves $\mathrm{Tr}_{\mathrm{blunt}}$ iff $\sum_{d,i} (K_i^{dc})^* K_i^{dc} = 1_c$ for all c. For an input state with $\mathrm{Tr}\,\rho_c > 0$, $T_{dc} = \mathrm{Tr}\,\Phi_{dc}(\rho_c)/\mathrm{Tr}\,\rho_c$ are the transition probabilities between sectors. They do not depend on ρ iff every $\sum_i (K_i^{dc})^* K_i^{dc}$ is a multiple of 1_c; by default the library uses the maximally mixed state of each input sector. The dual with respect to tr_w on both sides is $\Phi^{*,w}(A)_c = \sum_{d,i} (w_d^{\mathrm{out}} / w_c^{\mathrm{in}}) (K_i^{dc})^* A_d K_i^{dc}$. For sector-preserving maps on one algebra the weights cancel and all three traces give the same dual.

```python
import torch
from torch_vn_algebra import TypeIAlgebra

alg = TypeIAlgebra([4, 4, 8], [4, 4, 8], complex_valued=False)  # M_4 + M_4 + M_8, real
B = 100_000                                          # Monte Carlo samples: one batch

def spectrum(dim, delta=0.6):                        # lambda_max = 1, Michelson contrast delta
    lo = (1 - delta) / (1 + delta)
    lam = lo + (1 - lo) * torch.rand(B, dim)
    lam[:, 0], lam[:, 1] = 1.0, lo
    return lam

X = alg.operator_from_eigenvalues(spectrum, batch_size=B, force_positive=True)
Y = alg.operator_from_eigenvalues(spectrum, batch_size=B, force_positive=True)
U = alg.random_unitary_operator(B)                   # Haar-orthogonal in every block
z = (X @ U @ Y).trace.abs() - (X @ Y).trace          # z = |Tr(XUY)| - Tr(XY)
```

Listing 1: The Monte Carlo of (2) in $M_4 \oplus M_4 \oplus M_8$ over $\mathbb{R}$: 10^5 samples in one batch. `operator_from_eigenvalues` draws independent Haar eigenvectors in every block, `trace` is $\mathrm{Tr}_{\mathrm{blunt}}$.

Tensor products and fusion of sectors. $\big(\bigoplus_c M_{k_c}\big) \otimes \big(\bigoplus_d M_{m_d}\big) = \bigoplus_{(c,d)} M_{k_c m_d}$ has sectors labelled by pairs. If the sectors carry additive charges q_c, q'_d, the operators that conserve only the *total* charge form the larger algebra

$$\mathcal{M}_1 \hat{\otimes} \mathcal{M}_2 = \bigoplus_q M_{n_q}, \qquad n_q = \sum_{q_c + q'_d = q} k_c m_d.$$

It contains $\mathcal{M}_1 \otimes \mathcal{M}_2$ and also operators such as $S_i^+ S_j^-$ that move charge between the factors. If distinct sectors of $\mathcal{M}_1$ carry distinct charges, the partial trace $(\mathrm{Tr}_2 X)_c = \sum_d \mathrm{Tr}_{m_d} X_{(c,d),(c,d)}$ still lands in $\mathcal{M}_1$. In both cases it is a map between sectors, and its dual with respect to $\mathrm{Tr}_{\mathrm{blunt}}$ is $A \mapsto A \otimes 1$.

Spin chains and dynamics. For L spins 1/2 with conserved S^z_{tot}, iterating the fused product of single sites $M_1 \oplus M_1$ (charges $0, 1$) gives $\mathcal{M} = \bigoplus_{N=0}^{L} M_{\binom{L}{N}}$, N the number of up spins. It contains every S^z_{tot}-conserving Hamiltonian, e.g. the XXZ chain $H = \sum_{\langle ij \rangle} \big[\frac{J}{2}(S_i^+ S_j^- + S_i^- S_j^+) + J\Delta_z S_i^z S_j^z\big] + \sum_i h_i S_i^z$. For $H = H^* \in \mathcal{M}$ and jump operators $L_j \in \mathcal{M}$ the Lindblad equation $\dot\rho = -i[H, \rho] + \sum_j \gamma_j (L_j \rho L_j^* - \frac{1}{2}\{L_j^* L_j, \rho\})$ preserves every sector, so the sector probabilities p_c are conserved. Jumps between sectors, such as particle loss, are maps between sectors as above.

4 Reproducing the earlier NumPy study

The Monte Carlo experiments of [9] (Sect. 5, a single block $M_n(\mathbb{R})$, $n \le 5$) and of [4] (several sectors) sample trace functionals such as

$$z = |\,\mathrm{Tr}(XUY)| - \mathrm{Tr}(XY), \qquad X, Y \ge 0, \quad U \in \mathcal{M} \text{ unitary}, \tag{2}$$

with $\mathrm{Tr} = \mathrm{Tr}_{\mathrm{blunt}}$, the operators normalised to $\lambda_{\max} = 1$ and their contrasts as parameters ("Exp. 1" in Section 8). Listing 1 is the complete library version: it declares a real algebra, samples 10^5 pairs with prescribed contrast and Haar-orthogonal eigenvectors in every block, and evaluates z for the whole batch.

C	k	NumPy loop	NumPy vectorised	library (CPU)	library (GPU)
1	2	160	2.49	1.50	124
16	16	2852	690	884	2000
32	16	5702	1380	1688	4005

Table 2: Time per Monte Carlo sample [μs] of (2) for C real sectors of size k, in double precision unless stated (median of up to five runs after a warm-up; full run on NVIDIA GeForce RTX 5090).

To check that nothing was lost in the re-implementation, the same computation was written three ways (`scripts/baselines.py`). The loop formulation draws one sample and one sector at a time with `scipy.stats.ortho_group`, as code written directly from the definition does. The hand-vectorised NumPy version stacks QR decompositions with the phase correction. The library version is Listing 1. The three versions sample the same distribution: their means of z differ by at most 1.2 standard errors.

Table 2 shows the times. Against the loop formulation the library gains about two orders of magnitude for blocks of size 2, where the Python overhead dominates, and about a factor of three for 16 or 32 blocks of size 16, where both spend their time in the same LAPACK routines. Against carefully vectorised NumPy the library is not systematically faster on a CPU, because both call the same kernels. Its value there is that the vectorised, correctly sampled, block-aware version is the default rather than an expert rewrite, and that the same code runs on a GPU and can be differentiated. GPU timings depend on the precision: the consumer GPU of this run executes FP64 at 1/64 of its FP32 rate, and in double precision it does not beat the CPU on this workload. Section 5 explains where the speed comes from, and Section 8 shows that the batched optimisation finds extremes of (2) that no amount of sampling reaches.

5 Design and implementation

5.1 Software stack and workflow

The library is pure Python on top of PyTorch [78] ($\geq$ 1.13) and NumPy [42] (Figure 1). Dense linear algebra goes through `torch.linalg`, i.e. LAPACK (MKL or OpenBLAS) on CPUs and cuSOLVER, cuBLAS and MAGMA on NVIDIA GPUs; sparse products use PyTorch's CSR tensors; automatic differentiation is PyTorch's reverse mode. The validation and paper scripts also use SciPy, pandas, Matplotlib and mpmath. The test suite runs in continuous integration on Python 3.8–3.13 and with the oldest supported versions (Python 3.8, PyTorch 1.13, NumPy 1.21). Every paper script records the device, the matmul precision and the PyTorch, CUDA and cuDNN versions in a JSON file next to its table.

A computation proceeds in four steps. (1) An algebra is declared by its sector sizes, and optionally sector charges, number field, precision and device (`TypeIAlgebra`, or a model such as `SpinChain` that creates it). (2) Objects are created in it: random operators with prescribed spectra, states, channels, Hamiltonians, or differentiable parametrisations. (3) They are combined lazily: products, functional calculus, channels, tensor products, time evolution. (4) Functionals are evaluated for the whole batch at once and reduced to statistics, or fed into an optimiser that updates the parameters of step (2). Every step keeps the batch dimension, so the same code serves one operator and 10^5 samples. Table 3 lists the implementation problems this raises.

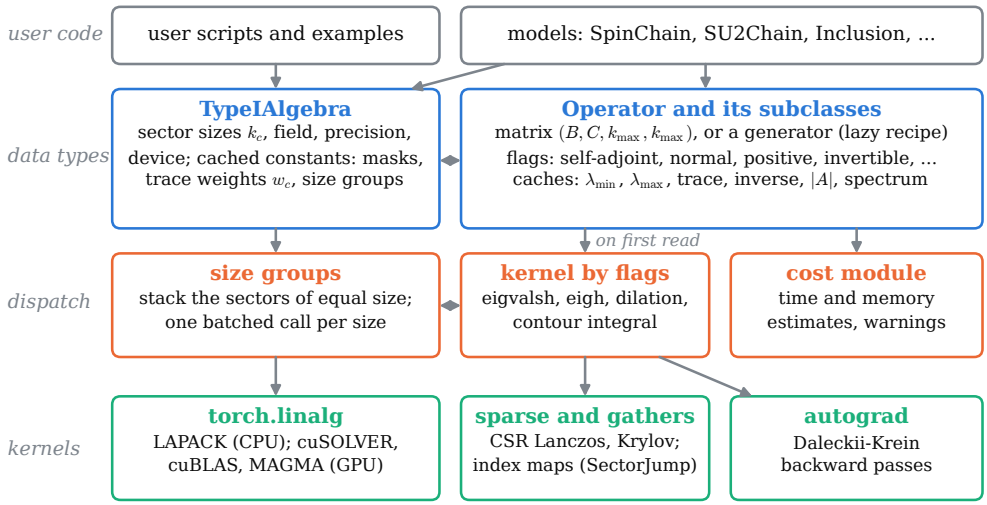


Figure 1: Architecture. User code and models create operators in a `TypeIAlgebra`; operators are matrices or lazy generators that carry property flags and caches. When a result is read, the dispatcher groups sectors of equal size and chooses the kernel from the flags; the cost module estimates time and memory first. The work ends in batched `torch.linalg` calls, sparse products or index gathers, and is differentiated by autograd with custom backward passes.

problem	solution in the library	section
(i) blocks of different size on batched kernels	padded $(B, C, k_{\max}, k_{\max})$ layout; one batched call per distinct block size; single sectors and sparse sector Hamiltonians where padding would dominate	5.2, 5.4
(ii) conventions between sectors	states stored as $\mathrm{Tr}_{\mathrm{blunt}}$-densities with explicit conversions; duals computed with the weights of the chosen trace	3
(iii) exact random ensembles	Mezzadri's QR algorithm with phase correction; $W^T W$ and $W^R W$ for COE/CSE; checked against Weingarten moments and spacing statistics	5.5, 7
(iv) singular gradients at degenerate spectra	Daleckii–Krein backward passes; parametrisations that build operators from their spectra	5.5
(v) extremes missed by sampling	batched multistart optimisation with exact contrast constraints	8
(vi) time and memory budgets	probing of batched kernels with time estimates; memory checked against the free device memory	5.7
(vii) field and precision	two switches per algebra, conversions, measured trade-offs	5.6

Table 3: Implementation problems and where they are resolved.

5.2 Data layout and objects

An operator is a tensor of shape $(B, C, k_{\max}, k_{\max})$: B samples, C sectors, block c in the top-left $k_c \times k_c$ corner and zeros elsewhere (Figure 2). Linear algebra is applied to the batch of active blocks, so one call to a batched LAPACK or cuSOLVER routine serves all samples. Padding costs memory when the sector sizes differ strongly: for a chain of 14 spins, 77 % of a padded operator is zeros. The library warns in that case, and `SpinChain` lets one work in single sectors $M_{\binom{L}{N}}$ without padding, which is what dense diagonalisation needs anyway. `Operator`, with the subclass `DensityMatrix` for states, `Channel` (sector preserving) and `InterSectorChannel` (between sectors or algebras) are

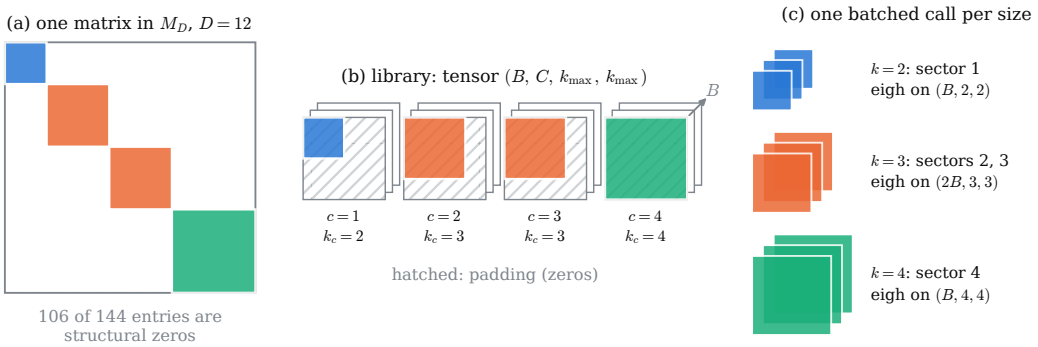


Figure 2: Data layout for $\mathcal{M} = M_2 \oplus M_3 \oplus M_3 \oplus M_4$. (a) As one block-diagonal matrix most entries are structural zeros. (b) The library stores a batch of B operators as a $(B, C, k_{\max}, k_{\max})$ tensor, each block in the top-left corner of its slice. (c) Decompositions stack the blocks of equal size, so the two sectors of size 3 and all samples go into one batched call.

module	content
`algebra`	`TypeIAlgebra`, `Operator`: random operators, unitary ensembles, traces, functional calculus
`states`	`DensityMatrix`, random and Gibbs states, Lüders rule, entropies, fidelity
`channels`	`Channel`, `InterSectorChannel`, `SectorJump`, Choi matrices, standard and random channels
`composite`	plain and fused (charge-conserving) tensor products, `kron`, partial traces
`chains`, `krylov`	`SpinChain`: $U(1)$ and momentum sectors, XXZ Hamiltonians with batched disorder, jumps between sectors; sparse sector Hamiltonians, batched Lanczos and Krylov
`dynamics`	exact Schrödinger/von Neumann evolution, RK4, Lindblad equation (RK4, completely positive splitting, $e^{t\mathcal{L}}$)
`functional`	functional calculus: self-adjoint, normal, singular values, holomorphic (contour); spectral cache
`optimize`, `cost`	parametrisations with prescribed contrast, batched multistart `extremize`; run-time and memory estimates
`inclusions`	`Inclusion`: inclusion matrices, conditional expectations, Markov trace, Jones, Watatani and Pimsner–Popa indices, basic construction, spin-chain tower
`decompose`, `su2`	numerical Wedderburn decomposition, commutants, real/complex/quaternionic types; `SU2Chain`: SU(2) multiplets, Clebsch–Gordan coefficients, Wigner–Eckart theorem
`divergences`, `jordan`	Petz and sandwiched Rényi divergences, recovery maps, L^p norms; Jordan product, quadratic representation, spin factors

Table 4: Modules of `torch_vn_algebra`.

thin wrappers around such tensors; `SectorJump` is a sparse jump operator for the Lindblad solvers (Section 5.4). Table 4 lists the modules.

5.3 Lazy evaluation and caches

Operators are recipes until used. An `Operator` holds either its matrix or a *generator*, a closure that computes it. Sums, products, adjoints, functions $f(A)$, channel images $\Phi(\rho)$ and maps between sectors return new operators that hold only a generator. The matrix is formed on the first access to `.matrix`: the generator runs, its result is stored, and the generator is dropped so that it no longer keeps the parent matrices alive. A long expression therefore costs exactly the products needed for

the requested result, and branches that are never read are never computed. The batch size of a lazy operator is known without computing it, so shapes and memory estimates are available before any work is done. Laziness defers and skips work but does not fuse kernels. Where fusion matters, the library uses direct formulas instead: $\mathrm{Tr}(\rho A) = \sum_{ij} \rho_{ij} A_{ji}$ in $O(k^2)$ per block without forming ρA, and the sector probabilities of a channel image as one contraction $\sum K_{mi} \rho_{ij} \overline{K_{mj}}$ without forming the image.

Properties travel with the recipe. Every operator carries flags (self-adjoint, normal, positive, invertible, projection) that are known, unknown, or checked. Operations set the flags that hold mathematically: sums of self-adjoint operators are self-adjoint, products of invertible operators invertible, and an operator built from its spectrum is self-adjoint, and positive if the spectrum is. Flags select the kernel. A self-adjoint operator uses the Hermitian eigensolver for its norm, trace norm, inverse and $|A|$, never an SVD, and $\lambda_{\max}$ of an operator built from its spectrum is read from the spectrum. An unknown flag is computed once, on demand, and stored. Flags passed by the user are trusted on eager operators and validated when a lazy operator is materialised, with one host synchronisation for all checks.

Caches. The extreme eigenvalues (from one eigenvalue call, without eigenvectors), the trace, the inverse and $|A|$ are cached on the operator. The eigendecomposition itself is kept only on request (`cache_spectrum()`, or the context `caching()`); afterwards every function of the operator, for instance e^{-itH} for many t, costs two matrix products. The cache is keyed by the identity and version counter of the stored tensor, so an in-place change invalidates it. The algebra caches its own constants on the device (block sizes, masks, identities, trace weights, index tensors of the size groups), so no Python list is copied to the GPU inside a loop.

5.4 Where the speed comes from

Three principles account for the speed, and Table 5 measures several of the decisions behind them by computing the same result with and without each. The alternatives are deliberately the simple ones a direct implementation would use, so the factors measure the value of each decision, not a comparison with other software.

The first principle is to give every kernel as much independent work as possible. Every operation acts on the whole batch, so a Monte Carlo study of 10^5 samples is one tensor program, not 10^5 calls. Decompositions are applied to the stacked active blocks of all sectors of equal size, one batched call per distinct size (Figure 2c); contiguous sectors are selected as views, and an unpadded algebra needs no scatter.

The second principle is to compute only what is read. One batched Hermitian eigensolver serves spectra, $f(A)$, $|A|$, singular values, polar decompositions and normal operators (Section 5.5); it is fast and, unlike batched SVD on GPUs, not limited in block size. Where only the spectrum is needed (extreme eigenvalues, contrasts, entropies), eigenvectors are not formed. Where the spectrum is known from the construction, no decomposition is run at all, and a cached eigendecomposition turns many functions of one operator into products.

The third principle is to keep the device busy and the memory small. Inner loops do not call `.item()` or branch on device values, so checks synchronise once per call, and the Lindblad steppers prepare constant propagators once per step size. Sparse structure is used where it exists. Sector Hamiltonians of spin chains are CSR matrices for batched Lanczos and Krylov solvers (Section 9). Jumps that map basis vectors to multiples of basis vectors ($S_i^{\pm}$, S_i^z, particle loss) are stored as `SectorJump`, with a target sector per sector and a target position and coefficient per basis vector.

mechanism	computation	library [ms]	without [ms]	speed-up
known spectrum	$\lambda_{\max}$, 256 operators in $M_{32}^{\oplus 8}$	2.25	28.70	13
eigenvalues only	spectra of the same operators	28.27	93.60	3.3
spectral cache	e^{-itA} for 50 values of t	3019.57	11717.58	3.9
size groups	spectra, 256 operators in $M_8^{\oplus 64}$	19.69	73.53	3.7
trace of a product	$\mathrm{Tr}(\rho A)$, 32 pairs in $M_{256}^{\oplus 2}$	16.62	41.75	2.5
sparse jumps	Lindblad, 8 qubits, S_i^- and S_i^z	354.63	1010.80	2.9

Table 5: The same result with and without one design decision (best of several runs, double precision, CPU with 4 threads; `scripts/architecture_bench.py`). "Without": eigenvalues by `eigvalsh` instead of the known spectrum; `eigh` instead of `eigvalsh`; one decomposition per time instead of the spectral cache; a Python loop over sectors; forming ρA; dense jumps between sectors. For the last row the eight jumps S_i^- take 48.4 MiB as dense maps and 0.12 MiB as `SectorJump`s, and the two evolutions agree to 4×10^{-15}; its time is dominated by the dense Hamiltonian, which both versions share.

The Lindblad solvers then apply $L\rho L^*$ as a weighted permutation of the entries of ρ, L^*ZL as a gather and L^*L as a diagonal, in $O(\sum_c k_c^2)$ operations against $O(C^2 k_{\max}^3)$ for a dense map between sectors. A batch of coefficients gives one jump per sample, so a sweep over noise rates is one batched evolution.

5.5 Random unitaries, functional calculus and derivatives

Random unitaries. Haar unitaries on $U(n)$ and $O(n)$ come from the QR decomposition of a matrix with i.i.d. complex (real) Gaussian entries after fixing the phases (signs) of $\operatorname{diag} R$ [14]; $SU(n)$ and $SO(n)$ follow by multiplying with $\det^{-1/n}$ or flipping the sign of one column. The circular orthogonal and symplectic ensembles are W^TW and W^RW with W Haar on $U(n)$ (n even for W^RW, W^R the quaternion dual) [15]. Section 7 checks all of them.

Functional calculus. The module `functional` provides $f(A)$ at four levels of generality, each with batched kernels and one call per block size [48]. (1) For self-adjoint A, $f(A) = Vf(\Lambda)V^*$ from the Hermitian eigensolver. (2) For normal $A = H + iK$, H and K commute, and the eigenvectors of $H + \alpha K$ diagonalise A for all but finitely many α. The library tries a few irrational values, refines nearly tied eigenvectors with a second coefficient and corrects the remaining residual to first order, so normal operators also need only the Hermitian eigensolver. (3) Singular values, $|A|$, the pseudo-inverse and the polar decomposition come from the Hermitian eigensolver of the dilation $\left(\begin{smallmatrix} 0 & A \\ A^* & 0 \end{smallmatrix}\right)$, whose eigenvalues are $\pm s_i$; batched SVD on GPUs is limited to blocks of size at most 32, the batched Hermitian eigensolver is not. (4) For arbitrary A and f holomorphic near the spectrum, $f(A) = (2\pi i)^{-1} \oint f(z)(z-A)^{-1} dz$ is evaluated by the trapezoidal rule on a circle around the spectrum [49], i.e. by batched LU solves, doubling the nodes until two results agree; an unconverged result raises an error instead of returning a wrong matrix.

Differentiation. All operations are differentiable. Gradients propagated naively through eigenvectors contain factors $1/(\lambda_i - \lambda_j)$ and fail at degenerate spectra, which are typical here (projections, symmetric and extremal operators). The functional calculus therefore has its own backward pass: by the Daleckii–Krein formula [50, 51], the derivative of f at self-adjoint A is $dF = V\,(L \circ V^* dA\, V)\, V^*$, with the Loewner matrix $L_{ij} = (f(\lambda_i) - f(\lambda_j))/(\lambda_i - \lambda_j)$ of divided differences, replaced by f' for coinciding eigenvalues, which is finite whenever f is differentiable on the spectrum. Backward passes

type	B/entry	MB/batch	generate	eigh f(A)	SVD |A|	channel	expm	opt. step
`float32`	4	16	1	1	1	1	1	1
`float64`	8	32	1.2	7.7	21	3.7	2.1	3.4
`complex64`	8	32	1.1	1.6	1.8	1.8	1.5	1.8
`complex128`	16	64	3.2	11	65	13	5.4	11

Table 6: Bytes per entry, memory of one batch [MB], and time relative to `float32` for $C = 16$ sectors of size $k = 32$ and a batch of 256 operators (full run on NVIDIA GeForce RTX 5090; float32 times per sample: generate: 918 μs, eigh f(A): 7.24 μs, SVD |A|: 11.4 μs, channel: 1.83 μs, expm: 4.96 μs, opt. step: 70.9 μs).

through the eigenvectors stay finite at degenerate spectra only for losses that do not mix a degenerate eigenspace [52], and they then miss the entries $f'(\lambda)$ inside the degenerate blocks. The same backward pass serves $\sqrt{A}$, $\log A$, A^p, $|A|$ (as $\sqrt{A^*A}$, with $L_{ij} = 1/(s_i + s_j)$) and inverses; normal A use complex divided differences, and the contour integral is differentiated through its solves.

5.6 Number field and precision

Every algebra has two switches, `complex_valued` and `precision` (`'single'` or `'double'`), giving `float32`, `float64`, `complex64` and `complex128`; everything built on the algebra inherits them. `alg.like(...)` creates the same sector structure with other numerics and `op.cast(alg2)` moves an operator there, for instance to explore in single precision and re-evaluate the interesting cases in double precision.

The field is part of the model. Real matrices change the symmetry group and the random ensembles, not only the speed. Haar matrices of a real algebra are orthogonal ($\mathbb{E}|U_{11}|^4 = 3/(n(n+2))$ instead of $2/(n(n+1))$, Table 9). Real symmetric and complex Hermitian random matrices have mean spacing ratios 0.5307 (GOE) and 0.5996 (GUE) [24, Table I], against 0.5313 and 0.5993 in our samples. Real arithmetic loses nothing when the problem is real, e.g. for ground states of Hamiltonians without complex hoppings, or for the supremum of $|\operatorname{Tr}(XUY)|$ over unitaries for real $X, Y \geq 0$, which is attained by an orthogonal matrix. Complex matrices are required for e^{-iHt} and Lindblad dynamics with a Hamiltonian (the library refuses to run them in a real algebra), for Haar unitaries, CUE/CSE and random pure states, and for systems without an antiunitary symmetry squaring to $+1$.

Precision. Single precision resolves about 10^{-7} relative to the largest entry, enough for Monte Carlo statistics and exploratory optimisation. Double precision is needed when the quantity of interest is small compared with the operators, as Table 7 shows: the entropy of an almost pure state is wrong by more than 100 % in single precision, and the round-off floor of a trace inequality at equality is several orders of magnitude higher.

Cost. An entry takes 4, 8, 8 or 16 bytes (`alg.bytes_per_operator(B)` reports the memory of a batch). A complex multiply–add costs about four real ones; for eigensolvers, and for double against single precision on CPUs, the measured ratios are smaller (Table 6). Data-centre GPUs execute FP64 at half the FP32 rate, consumer GPUs at 1/32–1/64; the full-size runs of this paper used a consumer GPU. FP32 products may run in the TF32 format on recent GPUs; PyTorch keeps true FP32 by default, the library does not change this, and the paper scripts record the setting. We recommend the smallest type that is exact for the model and the precision that the size of the effect requires.

type	$\max\|UU^*-1\|$	$\|\sqrt{A}^2-A\|/\|A\|$	rel. error $S(\rho)$, $\rho\approx$ pure	$\|z\|$ at equality
`float32`	6.6×10^{-7}	2.0×10^{-6}	2.0×10^{0}	5.3×10^{-4}
`float64`	1.3×10^{-15}	4.0×10^{-15}	6.6×10^{-9}	1.4×10^{-12}
`complex64`	8.3×10^{-7}	2.3×10^{-6}	2.3×10^{0}	2.0×10^{-4}
`complex128`	1.6×10^{-15}	4.5×10^{-15}	8.9×10^{-9}	1.8×10^{-12}

Table 7: Accuracy of the same computations in the four storage types: Haar unitaries ($n = 64$), SVD square root, entropy of $\rho = (1-\varepsilon)|0\rangle\langle 0| + \varepsilon\, 1/n$ with $\varepsilon = 10^{-7}$, and the round-off floor of $z = \mathrm{Tr}(Y|X|Y) - \mathrm{Tr}\,|YXY|$ for commuting X, Y (exactly 0); full run on NVIDIA GeForce RTX 5090.

5.7 Safeguards

Batched computations make it easy to request hours of work or more memory than the device has. The `cost` module therefore times the first few items of every large batched decomposition and matrix exponential and warns with the expected total time when it exceeds a threshold (10 s by default). It compares the estimated peak memory of large allocations with the free device memory, warning above 50 % and raising `InsufficientMemoryError` above 90 %. After the second step of a time-stepping or optimisation loop it warns with the expected duration. All thresholds can be changed, and the checks can be switched off locally.

6 Usage by example

The repository contains short complete programs in `man/article/listings`; the test suite runs them, so they stay in sync with the code. Each runs on a CPU in under half a minute and uses a GPU when available. Table 8 lists the nine introductory examples; programs of later sections are described where they are used.

Several examples check exact statements. In `ex3`, the Kraus operators $\sqrt{1-gN}$, $\sqrt{g}\,a_1$, $\sqrt{g}\,a_2$ ($0 \le g \le \frac{1}{2}$) map sector N to N and $N-1$, and the classical part is the transition matrix of a pure death process, $T = \left(\begin{smallmatrix} 1 & g & 0 \\ 0 & 1-g & 2g \\ 0 & 0 & 1-2g \end{smallmatrix}\right)$ (columns: from $N = 0, 1, 2$). A unital positive map cannot increase the Michelson contrast, while the non-unital loss channel increases it for most samples. In `ex7` the ground state moves through the sectors until the chain is fully polarised above the saturation field $h_s = 2J$; `ex9` is the starting point of Section 11.

7 Validation against exact results

Table 9 compares Monte Carlo estimates (double precision) with exact values. The rows cover fourth moments of Haar unitaries and orthogonal matrices, and for traceless involutions $A = B$ and Haar U the value $\mathbb{E}\,\tau\big((AUBU^*)^2\big) = -1/(n^2-1)$ of the Weingarten calculus [20]. They also cover Page's formula for $\langle S(\rho_A)\rangle$ [16, 17, 18] and the mean purity $(k+r)/(kr+1)$ of random states of rank $r \le k$ [19, 21]. The last column is the deviation in standard errors; for a correct sampler these should look like standard normal deviates. For the circular ensembles the Wigner surmise only approximates the exact spacing distribution, so no z-score is given; Figure 3 shows the distributions, with L^1 distances 0.022, 0.007 and 0.006 for $\beta = 1, 2, 4$ from the surmise and level repulsion $P(s) \sim s^\beta$. The spontaneous-emission example agrees with the analytic solution to 2.2×10^{-16} (exact exponential) and 2.1×10^{-8} (RK4). The three traces satisfy their defining identities on random operators to round-off (Table 10), and $\mathrm{Tr}_{\mathrm{norm}} = C\,\tau_{\mathrm{vN}}$ holds to 9.0×10^{-16}.

program	what it shows	key API
`listings/ex1_random_operators.py`	10^4 positive operators in $M_2 \oplus M_3 \oplus M_4$ with contrast 0.6 and Haar-random eigenvectors in every sector; the three traces and spectral functions	`TypeIAlgebra`, `operator_from_eigenvalues`, `michelson_contrast`, `Tr_blunt`, `tau_vN`, `apply_function`
`listings/ex2_states_measurement.py`	Lüders conditioning on a projective measurement moves all probability into one sector; expectation values do not depend on the trace that represents the state	`states.random_density_matrix`, `condition_on`, `sector_probabilities`, `density`
`listings/ex3_channels.py`	particle loss of two fermionic modes as a map between the sectors $N = 0, 1, 2$; unital and non-unital channels and the contrast	`InterSectorChannel.from_blocks`, `transition_matrix`, `random_mixed_unitary_channel`
`listings/ex4_dynamics.py`	spontaneous emission of a two-level atom by RK4 and by the exponential of the generator, against $P_e(0)e^{-\gamma t}$ and $\lvert\rho_{eg}(0)\rvert e^{-\gamma t/2}$	`dynamics.lindblad_evolve`, `dynamics.lindblad_channel`
`listings/ex5_page_entropy.py`	mean entanglement entropy of random pure states on $\mathbb{C}^m \otimes \mathbb{C}^n$ against Page's formula [16]	`tensor_product`, `partial_trace`, `states.von_neumann_entropy`
`listings/ex6_inequality_search.py`	Haar sampling against batched gradient ascent for $\sup_U(\lvert\operatorname{Tr}(XUY)\rvert - \operatorname{Tr}(XY)) = \lVert YX\rVert_1 - \operatorname{Tr}(XY)$ (Section 8)	`random_unitary_operator`, `expm`, `trace_norm`
`listings/ex7_spin_chain.py`	Heisenberg ring of ten spins: the sector of the ground state in a uniform field, the mean level-spacing ratio of 100 disorder realisations in one batch, entanglement entropies	`SpinChain`, `random_field_heisenberg`, `level_spacing_ratio`, `entanglement_entropy`
`listings/ex8_large_and_open_chains.py`	Lanczos ground state of a sector of dimension 12870, Krylov evolution of a domain wall, momentum sectors, particle loss through jumps between sectors	`xxz_sparse`, `krylov.ground_state`, `krylov.evolve`, `xxz_momentum`
`listings/ex9_qaoa_xy_mixer.py`	XY mixer on a ring of six qubits with amplitude damping, dephasing and post-selection on the Hamming weight	`SpinChain.xxz`, `lowering`, `algebra.central`, `condition_on`

Table 8: The introductory programs in `man/article/listings` of the repository.

8 From sampling to optimisation

The NumPy study [9, 4] sampled three trace functionals:

$$\begin{aligned}
&\text{Exp. 1:} \quad z = |\operatorname{Tr}(XUY)| - \operatorname{Tr}(XY), && X, Y \geq 0,\ U \in \mathcal{M} \text{ unitary},\\
&\text{Exp. 2:} \quad z = \operatorname{Tr}|XY| - \operatorname{Tr}(|X|\,|Y|), && X = UX_0,\ Y = VY_0,\ X_0, Y_0 \geq 0,\ U, V \in \mathcal{M} \text{ unitary},\\
&\text{Exp. 3:} \quad z = \operatorname{Tr}(Y|X|Y) - \operatorname{Tr}|YXY|, && X = X^*,\ Y \geq 0,
\end{aligned}$$

with $\operatorname{Tr} = \operatorname{Tr}_{\text{blunt}}$, operators normalised to $\lambda_{\max} = 1$, and the contrasts of the operators as parameters (of $|X|$ and $|Y|$ for Exp. 2 and 3). Two observations motivate a different approach.

Sampling misses the extremes. For fixed X, Y the supremum in Exp. 1 is known exactly, $\sup_U z = \lVert YX\rVert_1 - \operatorname{Tr}(XY) > 0$. The best of 1 000 000 Haar samples falls short of it for every k, misses the violation for all pairs from $k = 4$ on, and moves further away as k grows (Table 11, Figure 4). Batched gradient ascent (128 starts, 500 steps) finds the supremum to a relative error of at most 3×10^{-5}. This is concentration of measure: the unitaries close to the maximiser form a set whose Haar measure decays rapidly with the dimension. The same effect explains why the Monte Carlo samples of [4] showed no violations for $k = 16$: it is a property of the sampling, not of the inequality.

Constrained optimisation. The natural objects are envelopes such as $\overline{z}(\delta) = \sup\{z : \Delta(X) = \delta\}$ and $\underline{z}(\delta) = \inf\{z : \Delta(X) = \delta\}$, taken over all remaining variables. The module `optimize` parametrises the constraint set exactly. The spectrum is $\lambda_{\max} = 1$, $\lambda_{\min} = (1-\delta)/(1+\delta)$, and the other eigenvalues

quantity	parameters	exact	Monte Carlo	std. error	z
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n = 2$	0.333333	0.333604	4.7×10^{-4}	+0.6
$\mathbb{E}\lvert U_{11}\rvert^2$, $U(n)$	$n = 2$	0.5	0.500277	4.6×10^{-4}	+0.6
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n = 4$	0.1	0.10028	2.2×10^{-4}	+1.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n = 8$	0.0277778	0.0277966	7.5×10^{-5}	+0.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n = 16$	0.00735294	0.00735803	2.3×10^{-5}	+0.2
$\mathbb{E}\lvert U_{11}\rvert^4$, $U(n)$	$n = 32$	0.00189394	0.00189625	6.2×10^{-6}	+0.4
$\mathbb{E}\lvert U_{11}\rvert^2$, $U(n)$	$n = 32$	0.03125	0.0312403	4.8×10^{-5}	-0.2
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n = 2$	0.375	0.375106	5.8×10^{-4}	+0.2
$\mathbb{E}\lvert U_{11}\rvert^2$, $O(n)$	$n = 2$	0.5	0.500155	5.6×10^{-4}	+0.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n = 4$	0.125	0.125085	3.1×10^{-4}	+0.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n = 8$	0.0375	0.0376573	1.3×10^{-4}	+1.2
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n = 16$	0.0104167	0.0104046	4.3×10^{-5}	-0.3
$\mathbb{E}\lvert U_{11}\rvert^4$, $O(n)$	$n = 32$	0.00275735	0.0027558	1.2×10^{-5}	-0.1
$\mathbb{E}\lvert U_{11}\rvert^2$, $O(n)$	$n = 32$	0.03125	0.0312731	6.7×10^{-5}	+0.3
$\mathbb{E}\,\tau((AUBU^*)^2)$	$n = 4$	-0.0666667	-0.0661548	6.3×10^{-4}	+0.8
$\mathbb{E}\,\tau((AUBU^*)^2)$	$n = 8$	-0.015873	-0.0159874	2.8×10^{-4}	-0.4
$\mathbb{E}\,\tau((AUBU^*)^2)$	$n = 16$	-0.00392157	-0.00414221	1.4×10^{-4}	-1.6
$\langle s^2 \rangle$, COE	$n = 64$	1.27324	1.28582	3.7×10^{-4}	–
$\langle s^2 \rangle$, CUE	$n = 64$	1.1781	1.18024	2.4×10^{-4}	–
$\langle s^2 \rangle$, CSE	$n = 64$	1.10447	1.10392	2.0×10^{-4}	–
$\langle S(\rho_A) \rangle$	$m = 2, n = 16$	0.646516	0.646516	1.6×10^{-4}	0.0
$\langle S(\rho_A) \rangle$	$m = 8, n = 16$	1.83367	1.83388	1.7×10^{-4}	+1.2
$\langle S(\rho_A) \rangle$	$m = 16, n = 16$	2.27487	2.27495	1.4×10^{-4}	+0.6
$\mathbb{E}\,\mathrm{Tr}\,\rho^2$	$k = 4, r = 2$	0.666667	0.666312	3.2×10^{-4}	-1.1
$\mathbb{E}\,\mathrm{Tr}\,\rho^2$	$k = 4, r = 4$	0.470588	0.470523	2.1×10^{-4}	-0.3
$\mathbb{E}\,\mathrm{Tr}\,\rho^2$	$k = 16, r = 4$	0.307692	0.307726	6.4×10^{-5}	+0.5
$\mathbb{E}\,\mathrm{Tr}\,\rho^2$	$k = 16, r = 16$	0.124514	0.124527	1.7×10^{-5}	+0.8

Table 9: Monte Carlo estimates versus exact values (for $\langle s^2 \rangle$ the Wigner surmise, an approximation, hence no z); samples: 400000 per Haar moment, 400000 per Weingarten row, 20000 matrices per circular ensemble, 50000 states per Page row, 100000 per purity row (full run on NVIDIA GeForce RTX 5090, PyTorch 2.14.1+cu130).

trace	error of tr(1)	$\max \lvert \mathrm{tr}(AB) - \mathrm{tr}(BA) \rvert$	$\max \lvert \mathrm{tr}(UAU^*) - \mathrm{tr}(A) \rvert$	$\min \mathrm{tr}(A^*A)/\lVert A \rVert_F^2$
$\mathrm{Tr}_{\mathrm{blunt}}$	0	4.0×10^{-15}	5.9×10^{-15}	1.000
$\mathrm{Tr}_{\mathrm{norm}}$	0	1.4×10^{-15}	1.9×10^{-15}	0.223
τ_{vN}	0	4.4×10^{-16}	6.1×10^{-16}	0.074

Table 10: Identities of the three traces on 2000 random non-Hermitian operators in $M_2 \oplus M_3 \oplus M_5$ (double precision): normalisation ($\sum_c k_c$, C, 1), tracial property, unitary invariance, and faithfulness; the last column is bounded below by $\min_c w_c$; full run on NVIDIA GeForce RTX 5090.

$\lambda_{\min} + (1 - \lambda_{\min})(1 + \sin\theta)/2$ in between; unlike a sigmoid, this reaches both ends at finite θ, where extremal spectra often lie. The positions of $\lambda_{\max}$ and $\lambda_{\min}$ are drawn per start, in the same sector or in different ones. The eigenvectors are $U = U_0 e^{G - G^*}$ with a Haar-random U_0 per start and a chart that is re-centred after every round. Discrete variables, such as the signs of the eigenvalues of a self-adjoint X (Exp. 3), are drawn per sector and improved by single sign flips between gradient rounds. Every iterate is admissible, and every grid point and start is one element of the batch, so each envelope is one batched optimisation.

Two implementation details matter. (a) In a real algebra a chart $U_0 e^{G - G^*}$ never leaves the

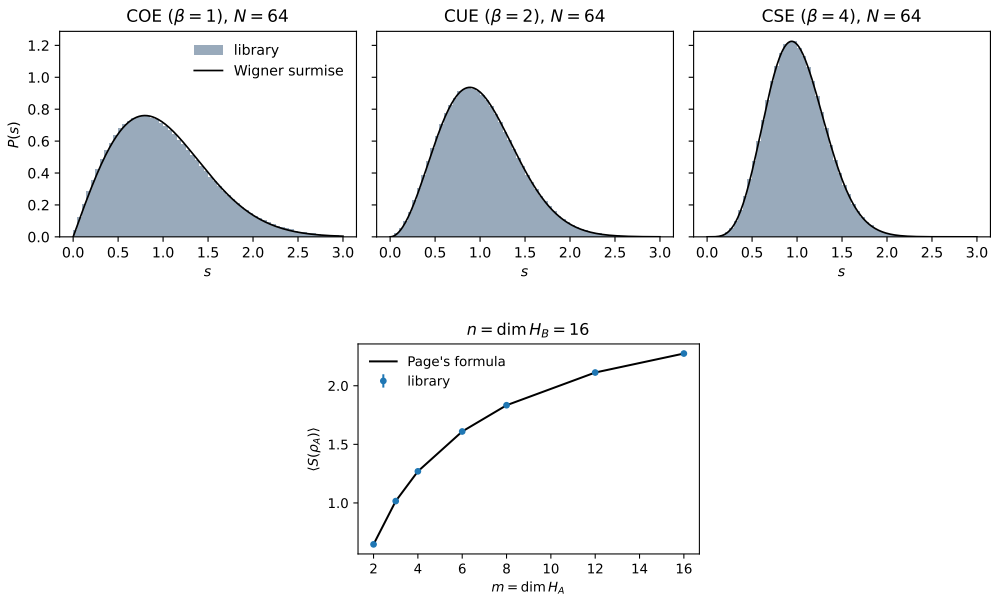


Figure 3: Top: nearest-neighbour spacings of COE, CUE and CSE eigenphases ($n = 64$, 20000 matrices per ensemble) against the Wigner surmise. Bottom: mean entanglement entropy of random pure states against Page's formula (50000 states per point) (error bars: two standard errors).

k	violation found		$\sup z/\operatorname{Tr} XY$	best sample	ascent rel. error		time [s]	
	sampling	ascent		$z/\operatorname{Tr} XY$	median	max	sampl.	ascent
2	14/16	16/16	7.1×10^{-4}	5.6×10^{-4}	1.2×10^{-8}	5.4×10^{-6}	64	2.8
4	0/16	16/16	2.3×10^{-3}	-0.12	4.4×10^{-8}	3.1×10^{-6}	66	2.8
8	0/16	16/16	5.6×10^{-3}	-0.48	2.9×10^{-7}	2.0×10^{-6}	70	2.8
16	0/16	16/16	6.7×10^{-3}	-0.72	2.4×10^{-6}	3.9×10^{-6}	88	2.7
32	0/16	16/16	8.9×10^{-3}	-0.86	7.4×10^{-6}	1.6×10^{-5}	1.8e+02	6.6
64	0/16	16/16	7.0×10^{-3}	-0.93	2.0×10^{-5}	2.8×10^{-5}	2.7e+02	11

Table 11: Exp. 1 for fixed random X, Y in $M_k(\mathbb{C})$ (eigenvalues uniform in $[0.2, 1.2]$, double precision): Haar sampling versus gradient ascent (medians over 16 pairs; full run on NVIDIA GeForce RTX 5090).

connected component of U_0, and an extremiser with determinant $+1$ in every sector is reached from a Haar-random U_0 only with probability 2^{-C}. Half of the starts therefore begin at a structured point (diagonal X, Y rotated in pairs of eigenvectors of X, two-level spectra). With Haar-random starts only, a first run for $C = 32$ sectors of size 2 fell short of the true envelopes by up to 35%; with structured starts it reaches them to a relative 10^{-5}–10^{-3}. (b) Where the objective has a kink at the maximiser (a trace norm at a degenerate point), the remaining gap decreases with the step size. Applied to the three functionals, the optimiser located their envelopes for sectors of every size; the closed forms it suggested are proved in the companion paper [1].

8.1 Rediscovering the data-processing thresholds of the Rényi divergences

A further test of the method is to ask it where the range of validity of a known inequality ends, without telling it. The module `divergences` evaluates the Petz divergences $D_\alpha(\rho\|\sigma) = \frac{1}{\alpha-1} \log \operatorname{Tr}(\rho^\alpha \sigma^{1-\alpha})$ [53] and the sandwiched divergences $\widetilde{D}_\alpha(\rho\|\sigma) = \frac{1}{\alpha-1} \log \operatorname{Tr}(\sigma^{\frac{1-\alpha}{2\alpha}} \rho\, \sigma^{\frac{1-\alpha}{2\alpha}})^\alpha$ [56, 57], batched and differen-

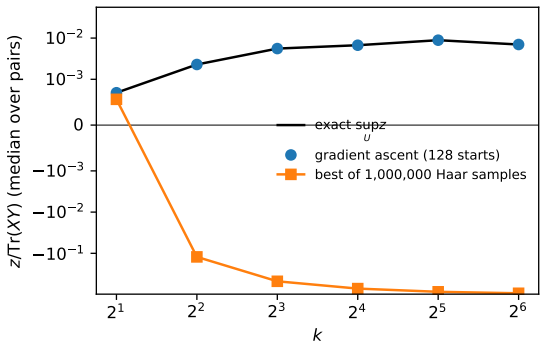


Figure 4: Best z found by sampling and by gradient ascent, relative to $\mathrm{Tr}(XY)$, against the exact supremum (symmetric log scale).

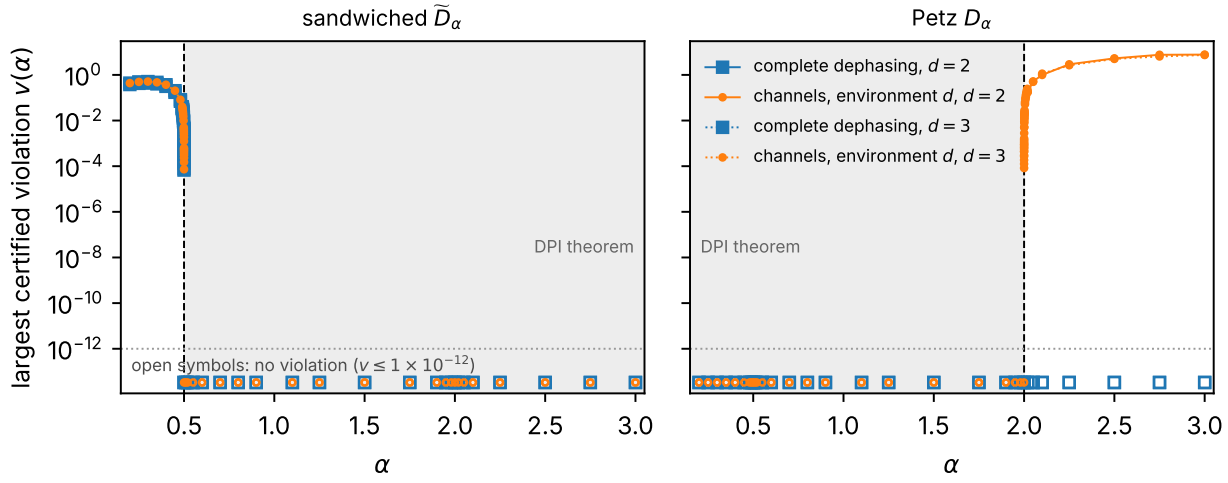


Figure 5: Largest certified violation $v(\alpha)$ for the sandwiched (left) and the Petz (right) divergences; dashed: the known thresholds; shaded: the range in which the data-processing inequality is a theorem.

tiable, with exact support conventions and as logarithms of sums over sectors; it also provides the Petz recovery map [54, 55] and the universal recovery map of [62]. The data-processing inequality $D(\Phi(\rho)\|\Phi(\sigma)) \le D(\rho\|\sigma)$ holds for the Petz divergences for $\alpha \in (0,1) \cup (1,2]$ [53] and for the sandwiched ones for $\alpha \ge \frac{1}{2}$ [56, 57, 59, 58], and fails in general outside these ranges [56, 60, 61]. `scripts/renyi_threshold.py` maximises the violation $v(\alpha) = D(\Phi(\rho)\|\Phi(\sigma)) - D(\rho\|\sigma)$ on a grid of $\alpha \in [0.2, 3]$, over states on M_d and over the complete dephasing and all channels with a d-dimensional environment, with all values of α and all starts in one batch. Every reported violation is attained at an explicit point and recomputed in mpmath with 50 significant digits, so a certified violation proves that the inequality fails at that α; the absence of a violation is evidence only. The sandwiched inequality is violated (certified) at $\alpha = 0.499980$ with $v = 7.8 \times 10^{-5}$, and no violation above 1×10^{-12} was found at $\alpha = 0.500000$; the Petz inequality is violated at $\alpha = 2.000039$ with $v = 1.3 \times 10^{-4}$, and no violation was found at $\alpha = 2.000000$. The thresholds are thus bracketed by $[0.499980, 0.500000]$ and $[2.000000, 2.000039]$, in agreement with $\frac{1}{2}$ and 2 ($d \in \{2, 3\}$, 64 starts and 600 steps per α; full run on CPU (4 threads)). Table 12 and Figure 5 show $v(\alpha)$ on the grid.

	sandwiched $\widetilde{D}_\alpha$		Petz D_α	
α	dephasing	channels	dephasing	channels
0.2	3.8×10^{-1}	3.8×10^{-1}	–	–
0.25	4.3×10^{-1}	4.3×10^{-1}	–	–
0.3	4.4×10^{-1}	4.4×10^{-1}	–	–
0.35	4.0×10^{-1}	4.0×10^{-1}	–	–
0.4	3.1×10^{-1}	3.1×10^{-1}	–	–
0.45	1.7×10^{-1}	1.7×10^{-1}	–	–
0.48	7.0×10^{-2}	7.0×10^{-2}	–	–
0.49	3.5×10^{-2}	3.5×10^{-2}	–	–
0.5	–	–	–	–
0.51	–	–	–	–
0.52	–	–	–	–
0.55	–	–	–	–
0.6	–	–	–	–
0.7	–	–	–	–
0.8	–	–	–	–
0.9	–	–	–	–
1.1	–	–	–	–
1.25	–	–	–	–
1.5	–	–	–	–
1.75	–	–	–	–
1.9	–	–	–	–
1.95	–	–	–	–
1.98	–	–	–	–
2	–	–	–	–
2.02	–	–	–	1.7×10^{-1}
2.05	–	–	–	5.2×10^{-1}
2.1	–	–	–	9.9×10^{-1}
2.25	–	–	–	2.9×10^{0}
2.5	–	–	–	5.3×10^{0}
2.75	–	–	–	7.6×10^{0}
3	–	–	–	7.8×10^{0}

Table 12: Largest certified violation $v(\alpha)$ of the data-processing inequality on M_2 ("–": no violation above the tolerance 1×10^{-12} found; full run on CPU (4 threads)). The theorems guarantee $v \le 0$ for $\alpha \ge \frac{1}{2}$ (sandwiched) and $\alpha \le 2$ (Petz).

9 Symmetric tensor products and spin chains

The basis of `SpinChain` is, by construction, that of the iterated fused product of single sites. Integrable chains provide exact results for every part of the module, and Table 13 lists those reproduced by `scripts/chains.py`; Table 16 and Figure 6 show some of them. Three entries need explanation. Peschel's formula gives the entanglement entropy of a block of the XX ground state from the eigenvalues ν of the restricted correlation matrix, $S = -\sum_\nu [\nu \ln \nu + (1-\nu)\ln(1-\nu)]$ [22], including the parity oscillations of open chains. Momentum sectors of periodic chains use the basis $|a,k\rangle = N_a^{-1/2} \sum_{j=0}^{L-1} e^{-ikj} T^j |a\rangle$ over orbit representatives a of the translation T (orbit length R_a, $N_a = L^2/R_a$; a contributes to k iff $kR_a \in 2\pi\mathbb{Z}$). The ground state of the antiferromagnetic Heisenberg ring has $k = 0$ for even and $k = \pi$ for odd $L/2$ (Marshall's sign rule). Blocks with $k \neq 0, \pi$ are complex in this basis even for real Hamiltonians, one of the cases where the field is fixed by the problem. With particle loss S_i^- on every site, any N-conserving Hamiltonian leaves the particle number a pure death process, so the sector distribution must be binomial.

For the Heisenberg chain with random fields $h_i \in [-W, W]$, the mean ratio of consecutive level

computation	exact reference	size	max. deviation
open XX chain, all 2^L levels	free fermions, $\varepsilon_k = J\cos\frac{\pi k}{L+1}$	$L = 12$	7.1×10^{-15}
Heisenberg ring, levels of sector $N < L/2$	reappear in sector $N+1$ (SU(2) multiplets)		2.9×10^{-15}
domain-wall melting, XX chain (dense)	single-particle $\langle n_i(t)\rangle = [e^{iht}C(0)e^{-iht}]_{ii}$	$L = 16$, dim 12870	4.9×10^{-15}
domain-wall melting (Krylov)	same	$L = 22$, dim 705432	1.7×10^{-14}
block entropy of the XX ground state	Peschel's formula	$L = 14$	3.8×10^{-12}
momentum sectors, $N = L/2$	union of spectra = spectrum of the sector	$L = 14$	3.6×10^{-14}
particle loss S_i^-, any N-conserving H	binomial $p_n(t) = \binom{N_0}{n} e^{-n\gamma t}(1-e^{-\gamma t})^{N_0-n}$		2.4×10^{-10}
Heisenberg ring ground states	Bethe ansatz ($L = 4$), exact diagonalisation	Table 16	nine digits

Table 13: Exact results reproduced by the chain module (`scripts/chains.py`; full run on NVIDIA GeForce RTX 5090).

model	sector dim	E_0 (Lanczos)	E_0 (exact)	difference	time [s]
XX open, $L = 16$	12870	-4.9189757237	-4.9189757237	1.8×10^{-15}	0.14
XX open, $L = 20$	184756	-6.1907449998	-6.1907449998	8.9×10^{-16}	0.07
XX open, $L = 24$	2704156	-7.4629855550	-7.4629855550	2.7×10^{-15}	0.56
Heisenberg ring, $L = 16$	12870	-7.1422963606	-7.142296361	$< 5 \times 10^{-10}$	0.059

Table 14: Batched Lanczos on sparse sector Hamiltonians (time includes building the sparse Hamiltonian): ground-state energies against free fermions and the tabulated Heisenberg value; full run on NVIDIA GeForce RTX 5090.

spacings in the middle of the spectrum crosses over from the GOE value 0.5307 to the Poisson value $2\ln 2 - 1 \approx 0.3863$ as W grows [23, 24] (Figure 6, right). At these sizes this is the finite-size crossover used as a spectral diagnostic in studies of many-body localisation, and the batch dimension holds the 400 disorder realisations per point.

Beyond dense matrices. Lanczos with full re-orthogonalisation and restarts, batched over Hamiltonians, reproduces free-fermion and Heisenberg ground-state energies in sectors up to the sizes of Table 14. For a single Hamiltonian on a CPU, ARPACK [44] (`scipy.sparse.linalg.eigsh`) is the reference. Our solver re-orthogonalises fully for robustness and is batched over Hamiltonians, which is what lets disorder averages and GPUs pay off (Table 15).

10 Subalgebras, inclusions and symmetries

Three modules relate algebras to each other and to the symmetries that produce them: `inclusions` treats a subalgebra $\mathcal{N} \subset \mathcal{M}$ with its conditional expectations and indices, `decompose` computes the algebra generated by given matrices or by a finite group, and `su2` uses the full SU(2) symmetry of spin-$\frac{1}{2}$ chains. Table 17 lists the identities against which the test suite checks them.

Inclusions and conditional expectations. Let $\mathcal{N} = \bigoplus_{c=1}^{C} M_{k_c}$ and $\mathcal{M} = \bigoplus_{d=1}^{D} M_{m_d}$. Every unital inclusion $\mathcal{N} \to \mathcal{M}$ is, up to an inner automorphism $\mathrm{Ad}\,u$ of $\mathcal{M}$, of the form $\iota(x)_d = \bigoplus_c x_c \otimes 1_{\lambda_{dc}}$. It is given by the inclusion matrix $\Lambda = (\lambda_{dc}) \in \mathbb{Z}_{\geq 0}^{D\times C}$, $m_d = \sum_c \lambda_{dc} k_c$, whose entry λ_{dc} is the

L	sector dim	Hamiltonians	`eigsh` [s]	library, GPU [s]	$\max\lvert\Delta E_0\rvert$
16	12870	1	0.0288	0.0552	6.2e-15
16	12870	20	0.621	0.114	4.3e-14
18	48620	1	0.115	0.0588	3.6e-15
20	184756	1	0.540	0.0642	1.1e-14

Table 15: Ground states of Heisenberg rings (one Hamiltonian) and random-field chains ($W = 3$, several Hamiltonians) in the sector $N = L/2$: ARPACK, one call per Hamiltonian, against the batched Lanczos solver; total time for all Hamiltonians (`scripts/baselines.py`; full run on NVIDIA GeForce RTX 5090).

L	dim ($N = L/2$)	E_0 (library)	E_0 (exact)	difference	E_0/L
4	6	-2.000000000	-2.000000000	$< 5 \times 10^{-10}$	-0.50000
6	20	-2.802775638	-2.802775638	$< 5 \times 10^{-10}$	-0.46713
8	70	-3.651093409	-3.651093409	$< 5 \times 10^{-10}$	-0.45639
10	252	-4.515446354	-4.515446354	$< 5 \times 10^{-10}$	-0.45154
12	924	-5.387390917	-5.387390917	$< 5 \times 10^{-10}$	-0.44895
14	3432	-6.263549534	-6.263549534	$< 5 \times 10^{-10}$	-0.44740
16	12870	-7.142296361	-7.142296361	$< 5 \times 10^{-10}$	-0.44639

Table 16: Ground-state energy of the Heisenberg ring $H = \sum_i \mathbf{S}_i \cdot \mathbf{S}_{i+1}$; exact values to nine digits; E_0/L approaches the Bethe-ansatz value $\frac{1}{4} - \ln 2 \approx -0.4431$ (full run on NVIDIA GeForce RTX 5090).

multiplicity of the summand c of $\mathcal{N}$ in the summand d of $\mathcal{M}$ [64]. `Inclusion` stores Λ and an optional batched unitary u; with the isometries $V_{dc,i}$ onto the copies of $\mathbb{C}^{k_c}$, ι is an `InterSectorChannel`. A faithful trace $\tau = \sum_d w_d \operatorname{Tr}_d$ on $\mathcal{M}$ restricts to $\mathcal{N}$ with weights $\nu = \Lambda^T w$, and

$$E(y)_c = \nu_c^{-1} \sum_{d,i} w_d \, V_{dc,i}^* u_d^* \, y_d \, u_d V_{dc,i}$$

is the unique τ-preserving conditional expectation onto $\mathcal{N}$, the adjoint of ι with respect to τ. For a connected inclusion the index is $[\mathcal{M} : \mathcal{N}] = \|\Lambda\|^2$ [63, 64], e.g. $(m/k)^2$ for $M_k \subset M_m$ and n for the diagonal subalgebra $\mathbb{C}^n \subset M_n$. The *Markov trace* is the trace whose weight vector is the Perron–Frobenius vector of $\Lambda\Lambda^T$; it is the trace for which the conditional expectation behaves as in a subfactor. The Watatani index is $\operatorname{Ind} E = \sum_j u_j u_j^*$ over a *quasi-basis*, a finite family $\{u_j\} \subset \mathcal{M}$ with $\sum_j u_j E(u_j^* y) = y$ for all y [66]. It is central, $(\operatorname{Ind} E)_d = (\Lambda\Lambda^T w)_d / w_d$, and equals $[\mathcal{M} : \mathcal{N}]\,1$ exactly for the Markov trace. The Pimsner–Popa constant $\lambda(E) = \max\{\lambda : E(y) \geq \lambda y \text{ for } y \geq 0\}$ equals $\big(\max_d \sum_c (\nu_c/w_d)\min(k_c, \lambda_{dc})\big)^{-1}$ [65, Theorem 6.1]. For subfactors of II_1 factors it is $[\mathcal{M} : \mathcal{N}]^{-1}$, but for Type I inclusions it differs; the library also computes the complete version $\max\{\lambda : E - \lambda\,\mathrm{id} \text{ completely positive}\} = 1/\|\operatorname{Ind} E\|$ [68, 67]. For $\mathbb{C} \subset M_n$ the two are $1/n$ and $1/n^2$. Basic construction, Jones projection, Jones tower and relative commutant are also available, all batched and differentiable (`listings/inclusions.py`).

The spin-chain tower. The algebras $A_L = \bigoplus_{N=0}^{L} M_{\binom{L}{N}}$ of Section 9 form the tower $A_L \subset A_{L+1}$, $x \mapsto x \otimes 1_2$, whose Bratteli diagram is Pascal's triangle. Its inclusion matrix is the incidence matrix of the path with $2L + 3$ vertices, so

$$[A_{L+1} : A_L] = \|\Lambda\|^2 = 4\cos^2\frac{\pi}{2L+4} \;\nearrow\; 4,$$

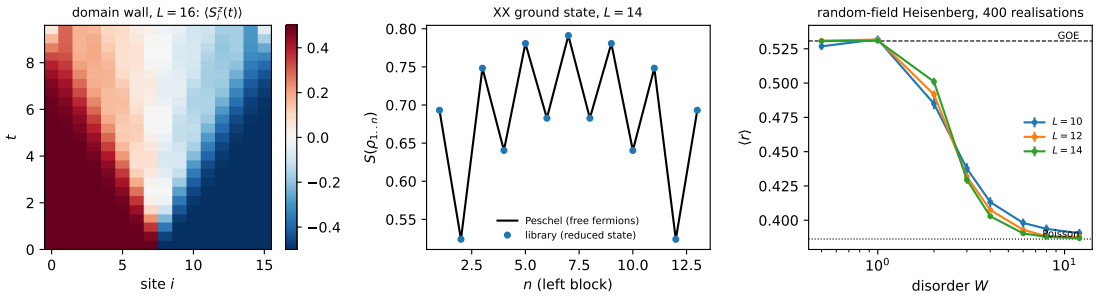


Figure 6: Left: $\langle S_i^z(t)\rangle$ after a domain-wall quench of the XX chain (light cone). Middle: entanglement entropy of the XX ground state, library vs. Peschel's formula. Right: mean level-spacing ratio of the random-field Heisenberg chain, with the GOE and Poisson values.

the values $4\cos^2(\pi/n)$, $n = 4, 6, 8, \ldots$, of Jones' discrete series below 4 [63].

From a symmetry to the algebra: Wedderburn decomposition. Given matrices $S_1, \ldots, S_J \in M_n(\mathbb{C})$, or a unitary representation π of a finite group, `decompose` returns a unitary V with

$$V^*\mathcal{A}V = \bigoplus_c M_{k_c}(\mathbb{C}) \otimes 1_{m_c}, \qquad V^*\mathcal{A}'V = \bigoplus_c 1_{k_c} \otimes M_{m_c}(\mathbb{C})$$

for the von Neumann algebra $\mathcal{A}$ generated by the matrices, together with maps into the algebras of the library. The algorithm combines two randomised methods of Murota and coauthors [69, 71]. The eigenspaces of a random Hermitian element of the commutant $\mathcal{A}'$ are, generically, irreducible $\mathcal{A}$-submodules. For a second random $Y \in \mathcal{A}'$ and orthonormal bases Q_i, Q_j of two submodules, $Q_i^* Y Q_j$ intertwines them; by Schur's lemma it vanishes for inequivalent submodules and is a multiple of a unitary otherwise, which gives the equivalence classes and aligns their bases. Random elements of $\mathcal{A}'$ come from the group average $|G|^{-1}\sum_g \pi(g) X \pi(g)^*$ or from the null space of $X \mapsto ([H_j, X])_j$. Every result is verified before it is returned (generators in the decomposed algebra, fresh commutant elements in its commutant, and $\sum_c m_c^2 = \dim \mathcal{A}'$, which together prove equality by the double commutant theorem); a failed check triggers a new draw. For real generators, `real_structure` classifies every summand as real, complex or quaternionic from the sign of $J_c\overline{J_c}$, which has a gap of 2, and `real_basis` returns an adapted real basis [70]. `listings/symmetry.py` recovers Schur–Weyl duality [72] for four spins.

SU(2) symmetry of spin chains. The Heisenberg Hamiltonian $H = \sum J_{ij}\, \mathbf{S}_i \cdot \mathbf{S}_j$ commutes with all of SU(2). On $(\mathbb{C}^2)^{\otimes L} = \bigoplus_S \mathbb{C}^{m_S} \otimes V_S$ the commutant is $\bigoplus_S M_{m_S}(\mathbb{C})$, $m_S = \binom{L}{L/2-S} - \binom{L}{L/2-S-1}$, and an invariant Hamiltonian is $\bigoplus_S H_S \otimes 1_{2S+1}$: every multiplet is computed once, and the blocks shrink from $\binom{L}{L/2+S}$ to m_S. `SU2Chain` realises $\mathbb{C}^{m_S}$ as the highest-weight vectors in the $U(1)$ sector $N = L/2 + S$, in the *Yamanouchi basis*: built by adding sites one at a time with exact Clebsch–Gordan coefficients (Racah's formula [73]) and labelled by the sequence of intermediate total spins. It is orthonormal by construction and expressed in the $U(1)$ basis, where the block of an invariant operator is $A_S = B_S^* A_N B_S$; spin correlations follow from reduced matrix elements (Wigner–Eckart theorem [74]). For $L = 14$ the largest block shrinks from 3432 to 1001 and the cubic cost of diagonalising all blocks by a factor of about 48; the bases themselves ($1.0 \cdot 10^8$ numbers for $L = 16$) limit dense SU(2) computations to about $L = 16$.

module	identities checked by the test suite
`chains`	S_i^z of `SpinChain(L)` equals $S_i^z \otimes 1$ formed with `kron` in the fused product of `SpinChain(L-1)` and a site
`inclusions`	E unital, completely positive, $E \circ \iota = \mathrm{id}$, bimodule property, adjoint of ι (three traces, Markov trace, random weights); Pimsner–Popa constants against brute force (bisection over rank-one y, Choi matrices); $e_{\mathcal{N}} y e_{\mathcal{N}} = E(y) e_{\mathcal{N}}$, $e_{\mathcal{N}} \in \mathcal{N}'$, Markov property of the basic construction; tower: path-graph spectrum and index for $L \leq 8$, embedding $x \otimes 1_2$, conditional expectation = normalised partial trace
`decompose`	hidden block structures under random unitaries; Fourier basis of the regular representation of $\mathbb{Z}_n$; Schur–Weyl duality on $(\mathbb{C}^2)^{\otimes L}$, $L \leq 6$ (same central projections, k_c and m_c exchanged); Kramers degeneracy in quaternionic sectors
`su2`	full spectra of open and periodic chains with field and next-nearest-neighbour couplings against dense diagonalisation ($L \leq 12$, 10^{-10}); ring energies of Table 16 ($L \leq 14$); Lieb–Mattis theorem [75]; the two degenerate singlets at the Majumdar–Ghosh point [76, 77]; Clebsch–Gordan coefficients against SymPy
`divergences`	dense references; independence of the trace convention; data processing inside the proven ranges; Petz's equality condition; the recovery bound of [62, Remark 2.2]

Table 17: Identities checked by the test suite for the modules of Sections 8.1–10.

11 Use case: noisy circuits with a conserved charge

Context. Quantum algorithms that preserve a charge offer a cheap test for errors. If the ideal evolution keeps the charge, a measurement of the charge at the end detects every error that changed it, and discarding those runs improves the accepted results. This *symmetry verification* [36, 37] is one of the standard error-mitigation techniques reviewed in [108]. Post-selection on the particle number was used, for instance, in Google's Hartree–Fock experiment [109]. Botelho et al. [107] proposed mid-circuit post-selection on Hamming-weight subspaces for QAOA. The XY mixers of the quantum alternating operator ansatz keep the Hamming weight [38, 105]; Streif et al. [106] derived the probability that such circuits stay in the feasible subspace under local depolarising noise and proposed codes that correct symmetry-breaking errors. In the language of open systems, the noisy register has a weak $U(1)$ symmetry in the sense of Buča and Prosen [110] and Albert and Jiang [111]. The Lindbladian commutes with conjugation by $e^{i\phi N}$, which says exactly that block-diagonal states stay block diagonal, i.e. that the dynamics never leaves $\mathcal{M} = \bigoplus_{N=0}^{n} M_{\binom{n}{N}}$. Without jumps that change N the symmetry is strong and every sector is conserved separately. In this language the measurement of the weight is a central projection P_w and post-selection is the Lüders conditioning on it, so the whole question is a computation in the algebra (Figure 7).

Model. A ring of n qubits evolves under the XY mixer $H = \frac{1}{4}\sum_{\langle ij\rangle}(X_iX_j + Y_iY_j) = \frac{1}{2}\sum_{\langle ij\rangle}(S_i^+S_j^- + S_i^-S_j^+)$ for a time $T = 2$ from $|1^w 0^{n-w}\rangle$, $w = n/2$. Three kinds of local noise act on every site: amplitude damping S_i^- (rate $\gamma_\downarrow$), thermal excitation S_i^+ ($\gamma_\uparrow$) and dephasing S_i^z (γ_φ). With ψ the noiseless state at time T we compute the acceptance probability $p = \mathrm{Tr}(P_w \rho(T))$, the fidelity $F = \langle\psi|\rho(T)|\psi\rangle$ and the fidelity after post-selection $F_{\mathrm{post}} = F/p$. Let $\Gamma_w = \gamma_\downarrow w + \gamma_\uparrow(n-w)$ be the total rate of leaving sector w and $I = \int_0^T \sum_i \big(\frac{1}{4} - \langle S_i^z\rangle^2_{\psi(t)}\big)\,dt$ the dephasing integral along the noiseless trajectory.

Proposition 1. *Let H (possibly time dependent) commute with N, and let the register start in a pure state of weight w.*

(a) The weight distribution $p_N(t) = \mathrm{Tr}(P_N\rho(t))$ is that of the classical birth–death process with death rate $\gamma_\downarrow N$ and birth rate $\gamma_\uparrow(n-N)$; it does not depend on H or γ_φ.

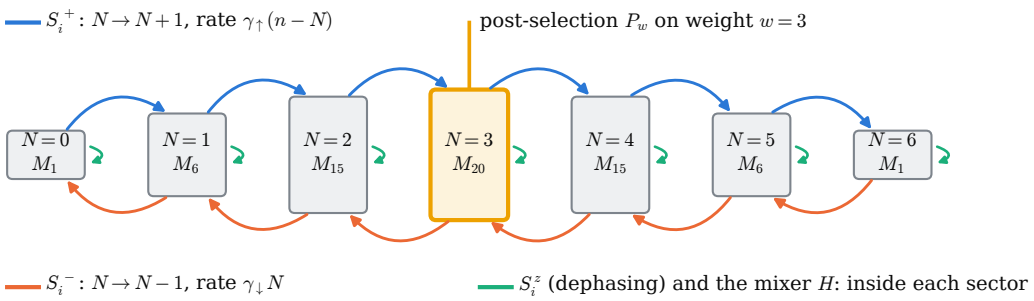


Figure 7: The sector ladder of a noisy register of $n = 6$ qubits. Sector N (Hamming weight) is the block $M_{\binom{n}{N}}$. Excitation S_i^+ moves up, amplitude damping S_i^- down, with total rates $\gamma_\uparrow(n-N)$ and $\gamma_\downarrow N$ that depend only on N. Dephasing and the mixer act inside the sectors, and post-selection projects onto the initial weight w.

(b) If $\gamma_\uparrow = 0$: $P_w\rho(T)P_w = e^{-\gamma_\downarrow wT}\rho_\varphi(T)$, with ρ_φ the solution for $\gamma_\downarrow = 0$. In particular $p = e^{-\gamma_\downarrow wT}$, and $F_{\text{post}} = 1$ if also $\gamma_\varphi = 0$.

(c) If $\gamma_\varphi = 0$: $0 \le 1 - F_{\text{post}} \le 1 - e^{-\Gamma_w T}/p = \gamma_\downarrow\gamma_\uparrow T^2\big[w(n-w) + \frac{n}{2}\big] + O\big((n\gamma T)^3\big)$, $\gamma = \max(\gamma_\downarrow, \gamma_\uparrow)$.

(d) If $\gamma_\downarrow = \gamma_\uparrow = 0$: $p = 1$ and $1 - F_{\text{post}} = 1 - F = \gamma_\varphi I + O(\gamma_\varphi^2)$.

(e) To first order in all rates, $1 - F = \Gamma_w T + \gamma_\varphi I$ and $1 - F_{\text{post}} = \gamma_\varphi I$; for $\gamma_\varphi > 0$ the gain $(1-F)/(1-F_{\text{post}})$ tends to $1 + \Gamma_w T/(\gamma_\varphi I)$ as the rates go to zero at fixed ratios.

Proof. Since $S_i^\pm P_N = P_{N\pm1} S_i^\pm$ and $\sum_i S_i^+ S_i^- = N$, $\sum_i S_i^- S_i^+ = n - N$ are central, $\dot{p}_N = \gamma_\downarrow[(N+1)p_{N+1} - Np_N] + \gamma_\uparrow[(n-N+1)p_{N-1} - (n-N)p_N]$; H and the dephasing jumps commute with P_N and do not contribute, which is (a). Expand $\rho(T)$ in the jumps $S_i^\pm$ (Dyson series). The no-jump generator differs from the Lindbladian with $\gamma_\downarrow = \gamma_\uparrow = 0$ by $-\frac{1}{2}\{\gamma_\downarrow N + \gamma_\uparrow(n-N), \cdot\}$, which on sector w is the scalar $-\Gamma_w$. Hence the part of $\rho(T)$ in sector w is $e^{-\Gamma_w T}\rho_\varphi(T)$ plus terms with $m \ge 1$ up and m down jumps. (b) Without up jumps only the first term remains. (c) For $\gamma_\varphi = 0$, $\rho_\varphi(T) = |\psi\rangle\langle\psi|$, and the remaining terms form $J \ge 0$ with $\operatorname{Tr} J = p - e^{-\Gamma_w T}$. Then $F_{\text{post}} = (e^{-\Gamma_w T} + \langle\psi|J|\psi\rangle)/p \ge e^{-\Gamma_w T}/p$. The leading terms of $\operatorname{Tr} J$ are one down and one up jump in either order, with probabilities $\gamma_\downarrow w\,\gamma_\uparrow(n-w+1)T^2/2$ and $\gamma_\uparrow(n-w)\,\gamma_\downarrow(w+1)T^2/2$. (d) Dephasing keeps the sector, and the first-order term gives $1 - F = \gamma_\varphi \int_0^T \sum_i (\langle (S_i^z)^2\rangle - \langle S_i^z\rangle^2)\,dt$ with $(S_i^z)^2 = \frac{1}{4}$. (e) Every first-order jump $S_i^\pm$ leaves sector w and contributes 0 to F, so $F = e^{-\Gamma_w T}(1 - \gamma_\varphi I) + O(\gamma^2)$; with post-selection only the dephasing term remains at first order, by (c) and (d). □

None of these statements is new in substance. Statement (b) is the standard reason why particle-number post-selection removes pure loss: N is central, so damping commutes with the post-selection. Statements (c)–(e) make explicit what remains. Thermal noise is suppressed from first to second order with a coefficient $w(n-w) + \frac{n}{2} = n(n+2)/4$ at half filling, and dephasing is not detected at all. We state the proposition because it fixes what an exact computation must reproduce and isolates the two numbers it cannot predict: the integral I, and the fraction of the bound in (c) that is actually lost.

Computation. Listing 2 shows the core of the computation for zero temperature. All noise models and rates form one batch: 28 settings per n (four noise models, seven rates between 10^{-3} and

```
import torch
from torch_vn_algebra import SpinChain, dynamics, states

n, w, T = 6, 3, 2.0
q = SpinChain(n, 'periodic', precision='double')   # sectors: Hamming weight N = 0..n
H = q.xxz(J=1.0, Delta=0.0)                         # XY mixer
rho0 = q.basis_state('111000')
psi = dynamics.von_neumann(rho0, H, [T])[0]         # noiseless reference state
g = torch.logspace(-3, -1, 7, dtype=torch.float64)  # seven damping rates: one batch
jumps = [q.sector_jump(i, 'lower').scaled(g.sqrt()) for i in range(n)]   # S^-_i
rho = dynamics.lindblad_evolve(rho0, H, jumps, [0.0, T], substeps=40, method='kraus')[-1]
P = q.algebra.central([float(N == w) for N in range(n + 1)])  # weight-w projection
post, p = rho.condition_on(P)                       # post-selection (Lueders rule)
F_post = states.trace_of_product(post, psi).real
print(p / torch.exp(-w * g * T) - 1, 1 - F_post)   # both ~ 0
```

Listing 2: Amplitude damping of a six-qubit XY mixer for seven rates in one batch, and post-selection on the Hamming weight (Proposition 1(b)): both printed deviations are at the level of the time-step error and of round-off.

					(b) T_1, zero temp.		(c) thermal	(d) dephasing		mixed, $\gamma \approx 0.022$	
n	$\binom{n}{w}$	$\sum_N \binom{n}{N}^2$	4^n	time [s]	err. p	$\lvert 1-F_{\text{post}}\rvert$	slope	I	$\frac{1-F_{\text{post}}}{\gamma_\varphi I}$	$1-F$	$1-F_{\text{post}}$
4	6	70	256	0	4e-7	9e-16	2.00	1.167	0.999	0.1126	0.0254
6	20	924	4096	1	6e-6	1e-15	2.00	1.380	0.999	0.1573	0.0307
8	70	12870	65536	18	2e-5	9e-16	2.00	1.421	0.999	0.1969	0.0328
10	252	184756	1048576	930	5e-5	3e-15	2.00	1.426	0.999	0.2340	0.0345

Table 18: Noisy XY mixer on a ring of n qubits, $T = 2$ (CPU, double precision). Dimensions: the weight sector, the algebra and a dense density matrix; time for all 28 settings. (b) Relative deviation of p from $e^{-w\gamma T}$ (the error of the second-order splitting) and $|1 - F_{\text{post}}|$, maxima over the rates. (c) Slope of $\log(1 - F_{\text{post}})$ against $\log\gamma$ for $\gamma_\uparrow = 0.2\gamma_\downarrow = 0.2\gamma$, at small γ. (d) The integral I and the ratio of the computed infidelity to its first-order value at $\gamma_\varphi = 10^{-3}$. Mixed: $\gamma_\downarrow = \gamma_\varphi = \gamma$, $\gamma_\uparrow = 0.1\gamma$ at $\gamma = 10^{-5/3} \approx 0.022$, without and with post-selection.

10^{-1}), with the rates folded into the coefficients of the jumps. One call of `lindblad_evolve` with the completely positive splitting (`method='kraus'`, 40 steps) evolves the batch. The jumps are `SectorJumps` (Section 5.4); as dense maps between sectors they would need $3n$ tensors of $(n+1)^2\binom{n}{n/2}^2$ entries, 2.3 GB each for $n = 12$. Table 18 and Figure 8 show the results for $n = 4, \dots, 10$ (script `scripts/noisy_xy_mixer.py`).

Results. The computation confirms the proposition in every case. At zero temperature the post-selected state agrees with the noiseless one to 3×10^{-15}, and the acceptance agrees with $e^{-w\gamma T}$ to 5×10^{-5}, the error of the time steps. For all noise models the acceptance agrees with the birth–death process of (a) to 3×10^{-4}. The thermal infidelity after post-selection has slope 2.00. For $n = 4, 6, 8, 10$ it is 0.73, 0.76, 0.81 and 0.84 times the bound of (c), independently of the rate. The complementary fraction is the overlap with ψ that a state retains after one up and one down jump. The dephasing infidelity agrees with $\gamma_\varphi I$ to 0.1%. The integral I saturates with n at fixed T (1.17 for $n = 4$, 1.43 for $n = 10$): by time T only the qubits near the two domain walls of $|1^w 0^{n-w}\rangle$ have moved, and dephasing hurts only where the state is not a product of S^z eigenstates. The detected error grows with the weight w and the undetected one does not, so post-selection gains more for larger registers. At $\gamma = 10^{-3}$ the gain for the mixed model is 4.75 for $n = 4$ and 8.61 for $n = 10$, within 1.3 % of

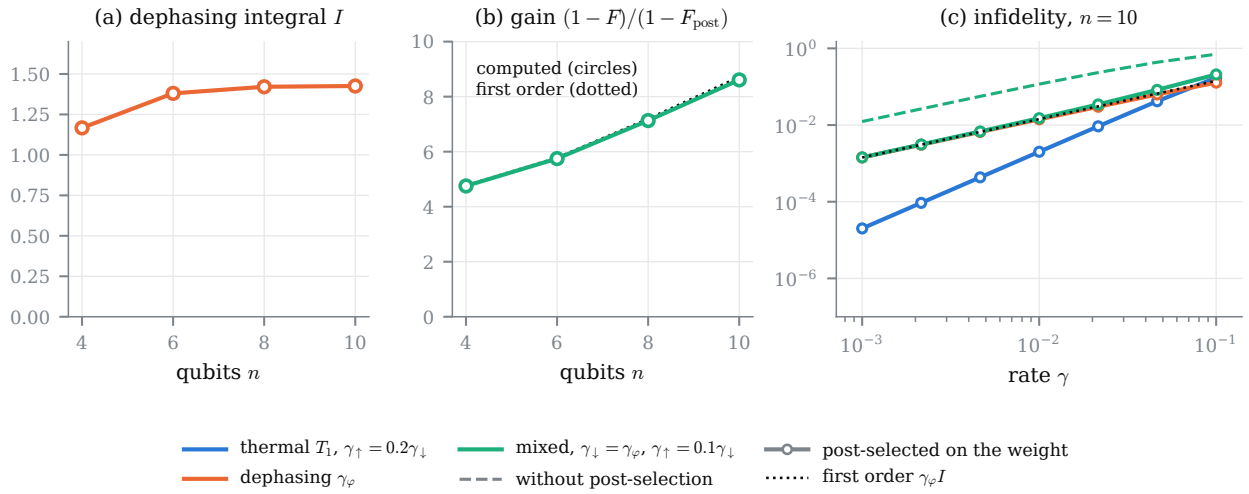


Figure 8: Noisy XY mixer, $T = 2$. (a) The dephasing integral I saturates with n. (b) The gain of post-selection for the mixed model at $\gamma = 10^{-3}$ grows with n and follows the first-order value $1+\Gamma_w T/(\gamma_\varphi I)$ of Proposition 1(e). (c) Infidelities for $n = 10$: post-selection suppresses thermal noise to second order and leaves dephasing, which follows $\gamma_\varphi I$; for zero-temperature damping (not shown) the post-selected state is exact.

the first-order value of (e). At $\gamma \approx 0.022$ it is 4.4 and 6.8 (Table 18). For algorithm design this quantifies the value of the symmetry: it protects against energy relaxation, not against dephasing, and the remaining error is predicted to first order by one integral over the noiseless trajectory.

What the computation adds, and what it does not cover. The computation gives exact values, up to the time-step error, for all noise models and rates in one batched run per register. It also gives the integral I and its saturation in n, the fraction of the thermal bound that is realised, and a gain that grows with n; none of these follows from the proposition alone. It does not model a QAOA circuit as practised. There is no phase separator, no layering, a single evolution time and a product initial state rather than a Dicke state. Phase separators are diagonal and commute with N, so the proposition still holds for layered circuits, but I and the gain depend on the circuit and would have to be recomputed. Two-qubit gate noise, readout errors and coherent errors are not modelled. Single-qubit depolarising noise does stay in the algebra: its Pauli-X and Y parts combine into $S^{\pm}$ jumps of equal rate, $\mathcal{D}[X] + \mathcal{D}[Y] = 2(\mathcal{D}[S^+] + \mathcal{D}[S^-])$, so it is the thermal model at $\gamma_\uparrow = \gamma_\downarrow$ plus dephasing. Noise that is not $U(1)$-covariant leaves the algebra: bit flips X_i alone, unequal X and Y rates, or coherent X rotations. For instance $X\rho X$ contains $S^+\rho S^+$, which maps the block (w, w) to the off-diagonal block $(w+1, w-1)$; such noise cannot be represented in (1) and needs the full M_{2^n}, which is a limitation of the approach. Finally, we make no claim of speed for this example. The run for $n = 10$ took 930 s on a CPU for 28 settings, dominated by dense products of the padded Hamiltonian (Section 14). A single density matrix of dimension 2^{10} is within easy reach of dedicated solvers [11, 80, 13]; comparisons with them are left for future work.

12 Applications in the companion papers

The library was built for, and used in, four further studies, which are reported with proofs, validation and figures in companion papers; here we only name the parts of the library they use.

How strongly trace characterisations fail (with S. A. Abed [1]). For 25 inequalities that characterise the trace, the violation by $\varphi_W = \mathrm{Tr}(W\,\cdot\,)$ is maximised over all operators at every contrast $\Delta(W)$, in one batched optimisation per inequality (`optimize.SpectrumParam`, `PositiveParam`, `UnitaryParam`; lattice operations $P \wedge Q$, $P \vee Q$ by functional calculus with Daleckii–Krein gradients), and the best points are recomputed in mpmath with 40 digits. The computed envelopes led to closed forms, proved in [1], that vanish at the trace to the orders δ, δ^2, δ^4 and δ^8.

The Connes–Størmer entropy of two MASAs [2]. The relative entropy $H(\Delta_n | U\Delta_n U^*)$ of the diagonal algebra and a rotated copy is a supremum over partitions of unity; with the conditional expectations of `Inclusion` and partitions parametrised by isometries, many unitaries and starts form one batch. For $n = 2$ the computation suggested a closed form, proved in [2]; for $n = 3, 4, 5$ it finds partitions that beat every orthonormal basis found.

A boundary time crystal under local dissipation [3]. Local emission and dephasing of N two-level systems commute with the permutations of the sites, so the state stays in the Schur–Weyl algebra $\bigoplus_j M_{2j+1} \otimes 1_{m_j}$, and the local dissipators are maps between its sectors (`PermutationalLindbladian`). With all sectors kept, the lifetimes of the oscillations follow for $N \leq 80$ without truncation, ten rates in one batch.

Symmetry-resolved entanglement in the XXZ chain [3]. The reduced state of a charge-conserving state is an operator of $\bigoplus_q M_{d_q}$, so resolved entropies and charge statistics are read off its blocks. For $L = 24$ one batched Lanczos run gives the ground states for eight anisotropies, and the charge statistics give the Luttinger parameter to about one per cent.

13 Performance

Table 19 reports the wall-clock time per sample of library operations (not of isolated kernels), in single precision, with batch $B = \min\bigl(4096,\, 2^{22}/(Ck^2)\bigr)$ (full run on NVIDIA GeForce RTX 5090; CPU with 16 threads; PyTorch 2.14.1+cu130, CUDA 13.0, cuSOLVER 12004). The operations are: generating random operators with prescribed spectra, $f(A)$ by eigendecomposition, $|A|$, the Michelson contrast, applying a channel, the Lindblad channel $e^{t\mathcal{L}}$, and one gradient step of the optimisation.

Library versions matter: batched Hermitian eigensolvers on GPUs cover all block sizes only in recent cuSOLVER and PyTorch releases, and batched SVD on GPUs is limited to blocks of size at most 32, which is why the library uses the Hermitian eigensolver wherever it can. Batched QR on GPUs factorises small matrices one at a time, so Haar sampling on GPUs uses a batched Gram–Schmidt orthonormalisation with reorthogonalisation (the same matrices as QR with the phase correction). On the CPU the library splits batched decompositions into one chunk per thread, so all cores are used.

GPUs need enough independent work per kernel launch: many samples, many sectors or large blocks, and matrix products rather than decompositions of small matrices. The design follows from this. Everything is batched, so the batch dimension supplies the parallel work, and the cost module warns before configurations that would be slow. The structure of the computation is also checked directly (`scripts/gpu_check.py` on a GPU, `tests/test_sync_free.py` on a CPU): the operations read no values back to the host, and their number of kernel launches does not grow with C at fixed total size. For workloads that are small or dominated by a single eigenproblem, the CPU, or SciPy, is the right tool, and the library runs there unchanged.

operation	k	C	batch	GPU [μs]	CPU [μs]	speedup
generate	4	1	4096	63.5	1.90	0.0299
f(A) eigh	4	1	4096	0.160	3.06	19.2
|A| SVD	4	1	4096	0.388	5.27	13.6
contrast	4	1	4096	0.0979	1.44	14.7
channel	4	1	4096	0.116	0.186	1.60
exp(tL)	4	1	4096	2.05	34.1	16.6
opt. step	4	1	4096	2.76	7.78	2.82
generate	4	16	4096	995	28.1	0.0283
f(A) eigh	4	16	4096	0.514	44.7	86.9
|A| SVD	4	16	4096	4.17	80.7	19.4
contrast	4	16	4096	0.358	22.0	61.5
channel	4	16	4096	1.34	4.53	3.38
exp(tL)	4	16	1024	49.2	556	11.3
opt. step	4	16	4096	9.54	80.2	8.41
generate	4	256	1024	15710	534	0.0340
f(A) eigh	4	256	1024	9.01	742	82.3
|A| SVD	4	256	1024	66.3	1283	19.4
contrast	4	256	1024	5.63	359	63.7
channel	4	256	1024	20.9	65.0	3.12
exp(tL)	4	256	64	7743	15365	1.98
opt. step	4	256	1024	133	1604	12.0
generate	16	1	4096	60.9	17.3	0.283
f(A) eigh	16	1	4096	0.206	23.8	116
|A| SVD	16	1	4096	0.780	44.5	57.0
contrast	16	1	4096	0.137	10.9	79.5
channel	16	1	4096	0.129	1.72	13.4
exp(tL)	16	1	64	1149	13608	11.8
opt. step	16	1	4096	3.61	74.3	20.6
generate	16	16	1024	1032	290	0.281
f(A) eigh	16	16	1024	1.76	419	238
|A| SVD	16	16	1024	10.7	746	69.6
contrast	16	16	1024	1.26	182	145
channel	16	16	1024	1.63	65.0	39.8
exp(tL)	16	16	4	79221	181417	2.29
opt. step	16	16	1024	34.9	1505	43.1
generate	16	256	64	18838	5937	0.315
f(A) eigh	16	256	64	78.0	6736	86.3
|A| SVD	16	256	64	172	11860	68.9
contrast	16	256	64	36.9	2871	77.7
channel	16	256	64	26.3	989	37.6
exp(tL)	16	256	1	2349043	3558629	1.51
opt. step	16	256	64	567	21657	38.2
generate	64	1	1024	62.0	329	5.30
f(A) eigh	64	1	1024	3.47	393	113
|A| SVD	64	1	1024	13.1	820	62.4
contrast	64	1	1024	2.48	211	85.0
channel	64	1	1024	0.859	71.2	82.9
opt. step	64	1	1024	34.2	1737	50.8
generate	64	16	64	1262	4995	3.96
f(A) eigh	64	16	64	59.2	6495	110
|A| SVD	64	16	64	211	12966	61.4
contrast	64	16	64	41.2	3424	83.0
channel	64	16	64	13.7	1054	76.8
opt. step	64	16	64	550	28154	51.2
generate	64	256	4	132631	103301	0.779
f(A) eigh	64	256	4	1753	99206	56.6
|A| SVD	64	256	4	3398	204740	60.2
contrast	64	256	4	933	54526	58.4
channel	64	256	4	217	12346	57.0
opt. step	64	256	4	8867	448973	50.6

Table 19: Wall-clock time per sample of library operations for C sectors of size k and the batch size shown (full run on NVIDIA GeForce RTX 5090, CPU with 16 threads) (single precision; "exp(tL)" uses a smaller batch because its superoperators have k^4 entries). With a GPU the table also gives the CPU time and the speedup CPU/GPU.

14 Limitations and roadmap

- *Layout.* Blocks are padded to $k_{\max}$. For very different sector sizes, such as the binomial sectors of spin chains, one works in single sectors, and `SU2Chain` builds the block of one total spin without padding; a bucketed layout for operators on all sectors is not implemented yet. Matrix

products of padded operators also multiply the padding: for the ten-qubit register of Section 11 the dense Hamiltonian part of a time step costs about four times $\sum_N k_N^3$, and it, not the jumps, dominates the run time.

- *Symmetries.* Abelian charges with an arbitrary fusion rule, lattice translations and, for spin-$\frac{1}{2}$ chains, SU(2) multiplets are supported; SU(2) with momentum, larger spins, reflections, spin flip and sparse solvers in SU(2) blocks are not. The Wedderburn decomposition uses dense linear algebra. Dense methods reach about 16–18 spins in the $U(1)$ sectors, the sparse solvers the half-filled sector of 24 spins (dimension $2.7 \cdot 10^6$).
- *Noise.* Only noise that commutes with the symmetry, in the weak sense of Section 11, can be represented; bit flips and other symmetry-breaking errors leave the algebra.
- *Optimisation.* The search is local, so envelopes are lower bounds for suprema. Gradients of the functional calculus are finite at degenerate spectra, but gradients through the eigenvectors returned by `eigh` are not, and second derivatives of $f(A)$ are correct at non-degenerate spectra only.
- *Divergences and scale.* For small α the sandwiched divergence of ill-conditioned states is not accurate (hence $\alpha \geq 0.2$ in Section 8.1). Choi matrices, superoperators and $e^{t\mathcal{L}}$ work with $k_c^2 \times k_c^2$ matrices and are limited to blocks of a few tens; RK4 and the splitting scale further.
- *Learning in the algebra.* Since a Hamiltonian or Lindblad evolution in the library is a differentiable composition of block-diagonal maps, a discretised evolution is a deep network whose layers are channels of the algebra, and its training is the continuous-depth idea of neural ordinary differential equations [112] with the constraints of quantum mechanics built in: unitary layers as in unitary recurrent networks [113], completely positive layers as in dissipative quantum neural networks [114], and conditional expectations as pooling. Equivariant linear layers are elements of the commutant, i.e. of an algebra (1), by Schur's lemma [40, 41], and networks with parameters in a C^*-algebra have been proposed in [115]. The library provides the primitives (parametrised unitaries and spectra, gradients through channels and through the dynamics; channels themselves are parametrised by Stinespring isometries in the scripts, not yet in the library) but no training layers or examples; this is the main direction of future work.
- *Scope.* Finite dimensions and Type I only; no tensor-network methods. Sparse methods exist only for sector Hamiltonians and for jumps that map basis vectors to basis vectors; other jumps between sectors are stored densely. Benchmarks against QuTiP, `dynamiqs`, QuantumToolbox.jl, QuSpin and XDiag are future work.

15 Availability and reproducibility

The code is available under the MIT licence at `https://github.com/andre-a-no/von_neumann_type_i`. The command `bash scripts/run_all.sh full cuda` regenerates every table and figure on a GPU; without arguments it runs a reduced version on a CPU in a few minutes. The script `scripts/gpu_check.py` checks a GPU, and `scripts/library_paper_schematics.py` draws the schematic figures. The tables state the device and mode they were produced with, and the JSON file next to each table records the commit of the code.

References

[1] S. A. Abed, I. A. Nikolaeva, A. A. Novikov, "How characterisations of the trace fail: sharp violation bounds in terms of Michelson contrasts", preprint (2026); source in `man/trace_envelopes` of the repository.

[2] I. Nikolaeva, A. Novikov, "The Connes–Størmer entropy of two maximal abelian subalgebras of M_2: an explicit variational formula, and partitions that beat bases in M_n", in preparation; source in `man/connes_stormer` of the repository.

[3] I. Nikolaeva, A. Novikov, "Exact symmetry sectors at work: finite-N lifetimes of a boundary time crystal under local dissipation and symmetry-resolved entanglement in the XXZ chain", in preparation; source in `man/btc_sre` of the repository.

[4] I. Nikolaeva, A. Novikov, "Finite-dimensional Type I von Neumann algebras in PyTorch: a GPU-accelerated framework for random block-diagonal operators", arXiv:2606.15882v1 (2026).

[5] L. T. Gardner, "An inequality characterizes the trace", Canad. J. Math. 31 (1979), no. 6, 1322–1328.

[6] D. Petz, J. Zemánek, "Characterizations of the trace", Linear Algebra Appl. 111 (1988), 43–52.

[7] A. A. Novikov, O. E. Tikhonov, "Characterization of central elements of operator algebras by inequalities", Lobachevskii J. Math. 36 (2015), no. 2, 208–210; arXiv:1502.01267.

[8] D. Virosztek, "Connections between centrality and local monotonicity of certain functions on C*-algebras", J. Math. Anal. Appl. 453 (2017), 221–226.

[9] S. A. Abed, I. A. Nikolaeva, A. A. Novikov, "Generalisation of Michelson contrast for operators and its properties", Lobachevskii J. Math. 45 (2024), no. 8, 3835–3848; arXiv:2010.10130.

[10] J. R. Johansson, P. D. Nation, F. Nori, "QuTiP 2: A Python framework for the dynamics of open quantum systems", Comput. Phys. Commun. 184 (2013), 1234–1240.

[11] N. Lambert et al., "QuTiP 5: The Quantum Toolbox in Python", Phys. Rep. 1153 (2026), 1–62; arXiv:2412.04705.

[12] A. Javadi-Abhari et al., "Quantum computing with Qiskit", arXiv:2405.08810 (2024).

[13] P. Guilmin, A. Bocquet, É. Genois, D. Weiss, R. Gautier, "Dynamiqs: an open-source Python library for GPU-accelerated and differentiable simulation of quantum systems", in preparation (2025); software version 0.3.7, `https://github.com/dynamiqs/dynamiqs` (accessed October 2026).

[14] F. Mezzadri, "How to generate random matrices from the classical compact groups", Notices Amer. Math. Soc. 54 (2007), 592–604.

[15] M. L. Mehta, *Random Matrices*, 3rd ed., Elsevier, 2004.

[16] D. N. Page, "Average entropy of a subsystem", Phys. Rev. Lett. 71 (1993), 1291–1294.

[17] S. K. Foong, S. Kanno, "Proof of Page's conjecture on the average entropy of a subsystem", Phys. Rev. Lett. 72 (1994), 1148.

[18] J. Sánchez-Ruiz, "Simple proof of Page's conjecture on the average entropy of a subsystem", Phys. Rev. E 52 (1995), 5653–5655.

[19] E. Lubkin, J. Math. Phys. 19 (1978), 1028.

[20] B. Collins, P. Śniady, "Integration with respect to the Haar measure on unitary, orthogonal and symplectic group", Commun. Math. Phys. 264 (2006), 773–795.

[21] K. Życzkowski, H.-J. Sommers, "Induced measures in the space of mixed quantum states", J. Phys. A 34 (2001), 7111–7125.

[22] I. Peschel, "Calculation of reduced density matrices from correlation functions", J. Phys. A 36 (2003), L205–L208.

[23] V. Oganesyan, D. A. Huse, "Localization of interacting fermions at high temperature", Phys. Rev. B 75 (2007), 155111.

[24] Y. Y. Atas, E. Bogomolny, O. Giraud, G. Roux, "Distribution of the ratio of consecutive level spacings in random matrix ensembles", Phys. Rev. Lett. 110 (2013), 084101.

[25] M. Takesaki, *Theory of Operator Algebras I*, Springer, 2002.

[26] G. C. Wick, A. S. Wightman, E. P. Wigner, "The intrinsic parity of elementary particles", Phys. Rev. 88 (1952), 101–105.

[27] S. D. Bartlett, T. Rudolph, R. W. Spekkens, "Reference frames, superselection rules, and quantum information", Rev. Mod. Phys. 79 (2007), 555–609.

[28] W. H. Zurek, "Decoherence, einselection, and the quantum origins of the classical", Rev. Mod. Phys. 75 (2003), 715–775.

[29] A. W. Sandvik, "Computational studies of quantum spin systems", AIP Conf. Proc. 1297 (2010), 135–338.

[30] P. Weinberg, M. Bukov, "QuSpin: a Python package for dynamics and exact diagonalisation of quantum many body systems part I: spin chains", SciPost Phys. 2 (2017), 003.

[31] M. Goldstein, E. Sela, "Symmetry-resolved entanglement in many-body systems", Phys. Rev. Lett. 120 (2018), 200602.

[32] J. C. Xavier, F. C. Alcaraz, G. Sierra, "Equipartition of the entanglement entropy", Phys. Rev. B 98 (2018), 041106.

[33] P. Zanardi, M. Rasetti, "Noiseless quantum codes", Phys. Rev. Lett. 79 (1997), 3306–3309.

[34] E. Knill, R. Laflamme, L. Viola, "Theory of quantum error correction for general noise", Phys. Rev. Lett. 84 (2000), 2525–2528.

[35] C. Bény, A. Kempf, D. W. Kribs, "Generalization of quantum error correction via the Heisenberg picture", Phys. Rev. Lett. 98 (2007), 100502.

[36] X. Bonet-Monroig, R. Sagastizabal, M. Singh, T. E. O'Brien, "Low-cost error mitigation by symmetry verification", Phys. Rev. A **98** (2018), 062339.

[37] S. McArdle, X. Yuan, S. Benjamin, "Error-mitigated digital quantum simulation", Phys. Rev. Lett. **122** (2019), 180501.

[38] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz", Algorithms 12 (2019), 34.

[39] F. J. Dyson, "The threefold way. Algebraic structure of symmetry groups and ensembles in quantum mechanics", J. Math. Phys. 3 (1962), 1199–1215.

[40] T. Cohen, M. Welling, "Group equivariant convolutional networks", Proc. ICML 2016, PMLR 48, 2990–2999.

[41] M. Finzi, M. Welling, A. G. Wilson, "A practical method for constructing equivariant multilayer perceptrons for arbitrary matrix groups", Proc. ICML 2021, PMLR 139, 3318–3328.

[42] C. R. Harris et al., "Array programming with NumPy", Nature 585 (2020), 357–362.

[43] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python", Nat. Methods 17 (2020), 261–272.

[44] R. B. Lehoucq, D. C. Sorensen, C. Yang, *ARPACK Users' Guide*, SIAM, 1998.

[45] M. Fishman, S. R. White, E. M. Stoudenmire, "The ITensor software library for tensor network calculations", SciPost Phys. Codebases 4 (2022).

[46] M. M. Rams, G. Wójtowicz, A. Sinha, J. Hasik, "YASTN: Yet another symmetric tensor networks; A Python library for Abelian symmetric tensor network calculations", SciPost Phys. Codebases 52 (2025); arXiv:2405.12196.

[47] J. Hauschild, F. Pollmann, "Efficient numerical simulations with tensor networks: Tensor Network Python (TeNPy)", SciPost Phys. Lect. Notes 5 (2018).

[48] N. J. Higham, *Functions of Matrices: Theory and Computation*, SIAM, 2008.

[49] L. N. Trefethen, J. A. C. Weideman, "The exponentially convergent trapezoidal rule", SIAM Rev. 56 (2014) 385–458.

[50] Ju. L. Daleckii, S. G. Krein, "Integration and differentiation of functions of Hermitian operators and applications to the theory of perturbations", Amer. Math. Soc. Transl. (2) 47 (1965) 1–30.

[51] R. Bhatia, *Matrix Analysis*, Graduate Texts in Mathematics 169, Springer, 1997.

[52] M. F. Kasim, "Derivatives of partial eigendecomposition of a real symmetric matrix for degenerate cases", arXiv:2011.04366 (2020).

[53] D. Petz, "Quasi-entropies for finite quantum systems", Rep. Math. Phys. 23 (1986), 57–65.

[54] D. Petz, "Sufficient subalgebras and the relative entropy of states of a von Neumann algebra", Commun. Math. Phys. 105 (1986), 123–131.

[55] D. Petz, "Sufficiency of channels over von Neumann algebras", Quart. J. Math. Oxford 39 (1988), no. 1, 97–108.

[56] M. Müller-Lennert, F. Dupuis, O. Szehr, S. Fehr, M. Tomamichel, "On quantum Rényi entropies: a new generalization and some properties", J. Math. Phys. 54 (2013), 122203.

[57] M. M. Wilde, A. Winter, D. Yang, "Strong converse for the classical capacity of entanglement-breaking and Hadamard channels via a sandwiched Rényi relative entropy", Commun. Math. Phys. 331 (2014), 593–622.

[58] R. L. Frank, E. H. Lieb, "Monotonicity of a relative Rényi entropy", J. Math. Phys. 54 (2013), 122201.

[59] S. Beigi, "Sandwiched Rényi divergence satisfies data processing inequality", J. Math. Phys. 54 (2013), 122202.

[60] M. Berta, O. Fawzi, M. Tomamichel, "On variational expressions for quantum relative entropies", Lett. Math. Phys. 107 (2017), 2239–2265.

[61] H. Zhang, "From Wigner–Yanase–Dyson conjecture to Carlen–Frank–Lieb conjecture", Adv. Math. 365 (2020), 107053.

[62] M. Junge, R. Renner, D. Sutter, M. M. Wilde, A. Winter, "Universal recovery maps and approximate sufficiency of quantum relative entropy", Ann. Henri Poincaré 19 (2018), 2955–2978.

[63] V. F. R. Jones, "Index for subfactors", Invent. Math. 72 (1983), 1–25.

[64] F. M. Goodman, P. de la Harpe, V. F. R. Jones, *Coxeter Graphs and Towers of Algebras*, MSRI Publications 14, Springer, 1989.

[65] M. Pimsner, S. Popa, "Entropy and index for subfactors", Ann. Sci. École Norm. Sup. (4) 19 (1986), 57–106.

[66] Y. Watatani, "Index for C^*-subalgebras", Mem. Amer. Math. Soc. 83 (1990), no. 424.

[67] M. Frank, E. Kirchberg, "On conditional expectations of finite index", J. Operator Theory 40 (1998), 87–111.

[68] M. Baillet, Y. Denizeau, J.-F. Havet, "Indice d'une espérance conditionnelle", Compositio Math. 66 (1988), 199–236.

[69] K. Murota, Y. Kanno, M. Kojima, S. Kojima, "A numerical algorithm for block-diagonal decomposition of matrix $*$-algebras with application to semidefinite programming", Japan J. Indust. Appl. Math. 27 (2010), 125–160.

[70] T. Maehara, K. Murota, "A numerical algorithm for block-diagonal decomposition of matrix $*$-algebras with general irreducible components", Japan J. Indust. Appl. Math. 27 (2010), 263–293.

[71] T. Maehara, K. Murota, "Algorithm for error-controlled simultaneous block-diagonalization of matrices", SIAM J. Matrix Anal. Appl. 32 (2011), 605–620.

[72] W. Fulton, J. Harris, *Representation Theory: A First Course*, Graduate Texts in Mathematics 129, Springer, 1991.

[73] G. Racah, "Theory of complex spectra. II", Phys. Rev. 62 (1942), 438–462.

[74] A. R. Edmonds, *Angular Momentum in Quantum Mechanics*, Princeton University Press, 1957.

[75] E. Lieb, D. Mattis, "Ordering energy levels of interacting spin systems", J. Math. Phys. 3 (1962), 749–751.

[76] C. K. Majumdar, D. K. Ghosh, "On next-nearest-neighbor interaction in linear chain. I", J. Math. Phys. 10 (1969), 1388–1398.

[77] C. K. Majumdar, D. K. Ghosh, "On next-nearest-neighbor interaction in linear chain. II", J. Math. Phys. 10 (1969), 1399–1402.

[78] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library", NeurIPS 2019.

[79] N. Shammah, S. Ahmed, N. Lambert, S. De Liberato, F. Nori, "Open quantum systems with local and collective incoherent processes: Efficient numerical simulations using permutational invariance", Phys. Rev. A 98 (2018), 063815.

[80] A. Mercurio, Y.-T. Huang, L.-X. Cai, Y.-N. Chen, V. Savona, F. Nori, "QuantumToolbox.jl: An efficient Julia framework for simulating open quantum systems", Quantum 9 (2025), 1866; doi:10.22331/q-2025-09-29-1866.

[81] Qiskit Aer, high-performance simulators for Qiskit, documentation at `https://qiskit.github.io/qiskit-aer/` (accessed October 2026).

[82] T. Jones, A. Brown, I. Bush, S. C. Benjamin, "QuEST and high performance simulation of quantum computers", Sci. Rep. 9 (2019), 10736; doi:10.1038/s41598-019-47174-9.

[83] H. Bayraktar et al., "cuQuantum SDK: A high-performance library for accelerating quantum science", in 2023 IEEE International Conference on Quantum Computing and Engineering (QCE), pp. 1050–1061; doi:10.1109/QCE57702.2023.00119; arXiv:2308.01999.

[84] NVIDIA, "cuDensityMat: a high-performance library for analog quantum dynamics computations", cuQuantum SDK documentation, `https://docs.nvidia.com/cuda/cuquantum/latest/cudensitymat/index.html` (accessed October 2026).

[85] V. Bergholm et al., "PennyLane: Automatic differentiation of hybrid quantum-classical computations", arXiv:1811.04968 (2018).

[86] P. Weinberg, M. Bukov, "QuSpin: a Python package for dynamics and exact diagonalisation of quantum many body systems. Part II: bosons, fermions and higher spins", SciPost Phys. 7 (2019), 020; doi:10.21468/SciPostPhys.7.2.020.

[87] A. Wietek, L. Staszewski, M. Ulaga, P. L. Ebert, H. Karlsson, S. Sarkar, L. Shackleton, A. Sinha, R. D. Soares, "XDiag: Exact diagonalization for quantum many-body systems", SciPost Phys. Codebases 70 (2026); doi:10.21468/SciPostPhysCodeb.70; arXiv:2505.02901.

[88] L. Devos, J. Haegeman, "TensorKit.jl: A Julia package for large-scale tensor computations, with a hint of category theory", arXiv:2508.10076 (2025).

[89] J. R. Gardner, G. Pleiss, D. Bindel, K. Q. Weinberger, A. G. Wilson, "GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration", in Advances in Neural Information Processing Systems 31 (NeurIPS 2018), pp. 7576–7586; arXiv:1809.11165.

[90] `linear_operator`: a PyTorch package for structured matrices, from the GPyTorch project, `https://github.com/cornellius-gp/linear_operator` (accessed October 2026).

[91] A. Potapczynski, M. Finzi, G. Pleiss, A. G. Wilson, "CoLA: Exploiting compositional structure for automatic and efficient numerical linear algebra", in Advances in Neural Information Processing Systems 36 (NeurIPS 2023); arXiv:2309.03060.

[92] M. Orlitzky, "Rank computation in Euclidean Jordan algebras", J. Symbolic Comput. 113 (2022), 181–192; doi:10.1016/j.jsc.2022.04.003; software at `https://codeberg.org/mjo/sage.d`.

[93] B. Nasmith, ALCO: tools for algebraic combinatorics, GAP package, version 1.1.2 (2025), `https://github.com/BNasmith/alco`.

[94] R. K. S. Hankin, "Jordan algebra in R", arXiv:2303.06062 (2023).

[95] P. A. Brooksbank, J. B. Wilson, "Computing isometry groups of Hermitian maps", Trans. Amer. Math. Soc. 364 (2012), 1975–1996; doi:10.1090/S0002-9947-2011-05388-2.

[96] E. de Klerk, C. Dobre, D. V. Pasechnik, "Numerical block diagonalization of matrix $*$-algebras with application to semidefinite programming", Math. Program. 129 (2011), 91–111; doi:10.1007/s10107-011-0461-3.

[97] F. Permenter, P. A. Parrilo, "Dimension reduction for semidefinite programs via Jordan algebras", Math. Program. 181 (2020), 51–84; doi:10.1007/s10107-019-01372-5.

[98] D. Rosset, F. Montealegre-Mora, J.-D. Bancal, "RepLAB: a computational/numerical approach to representation theory", in *Quantum Theory and Symmetries*, CRM Series in Mathematical Physics, Springer, 2021, pp. 643–653; doi:10.1007/978-3-030-55777-5_60; arXiv:1911.09154.

[99] M. Kaluba, T. Weisser, SymbolicWedderburn.jl, Julia package, `https://github.com/kalmarek/SymbolicWedderburn.jl` (accessed October 2026).

[100] P. J. Goulart, Y. Chen, "Clarabel: An interior-point solver for conic programs with quadratic objectives", Math. Program. Comput. (2026); doi:10.1007/s12532-026-00320-7.

[101] C. Coey, L. Kapelevich, J. P. Vielma, "Conic optimization with spectral functions on Euclidean Jordan algebras", Math. Oper. Res. (2023); doi:10.1287/moor.2022.1324.

[102] S. Morrison, N. Snyder, "Subfactors of index less than 5, Part 1: The principal graph odometer", Commun. Math. Phys. 312 (2012), 1–35; doi:10.1007/s00220-012-1426-y.

[103] V. F. R. Jones, S. Morrison, N. Snyder, "The classification of subfactors of index at most 5", Bull. Amer. Math. Soc. 51 (2014), 277–327; doi:10.1090/S0273-0979-2013-01442-3.

[104] A. Dedecker, J. Loreaux, "The continuous functional calculus in Lean", arXiv:2501.15639 (2025).

[105] Z. Wang, N. C. Rubin, J. M. Dominy, E. G. Rieffel, "XY mixers: Analytical and numerical results for the quantum alternating operator ansatz", Phys. Rev. A 101 (2020), 012320; doi:10.1103/PhysRevA.101.012320.

[106] M. Streif, M. Leib, F. Wudarski, E. Rieffel, Z. Wang, "Quantum algorithms with local particle-number conservation: Noise effects and error correction", Phys. Rev. A 103 (2021), 042412; doi:10.1103/PhysRevA.103.042412.

[107] L. Botelho, A. Glos, A. Kundu, J. A. Miszczak, Ö. Salehi, Z. Zimborás, "Error mitigation for variational quantum algorithms through mid-circuit measurements", Phys. Rev. A 105 (2022), 022441; doi:10.1103/PhysRevA.105.022441.

[108] Z. Cai, R. Babbush, S. C. Benjamin, S. Endo, W. J. Huggins, Y. Li, J. R. McClean, T. E. O'Brien, "Quantum error mitigation", Rev. Mod. Phys. 95 (2023), 045005; doi:10.1103/RevModPhys.95.045005.

[109] Google AI Quantum and Collaborators (F. Arute et al.), "Hartree-Fock on a superconducting qubit quantum computer", Science 369 (2020), 1084–1089; doi:10.1126/science.abb9811.

[110] B. Buča, T. Prosen, "A note on symmetry reductions of the Lindblad equation: transport in constrained open spin chains", New J. Phys. 14 (2012), 073007; doi:10.1088/1367-2630/14/7/073007.

[111] V. V. Albert, L. Jiang, "Symmetries and conserved quantities in Lindblad master equations", Phys. Rev. A 89 (2014), 022118; doi:10.1103/PhysRevA.89.022118.

[112] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, "Neural ordinary differential equations", Advances in Neural Information Processing Systems 31 (2018), 6571–6583.

[113] M. Arjovsky, A. Shah, Y. Bengio, "Unitary evolution recurrent neural networks", Proc. 33rd Int. Conf. Machine Learning, PMLR 48 (2016), 1120–1128.

[114] K. Beer, D. Bondarenko, T. Farrelly, T. J. Osborne, R. Salzmann, D. Scheiermann, R. Wolf, "Training deep quantum neural networks", Nat. Commun. 11 (2020), 808.

[115] Y. Hashimoto, Z. Wang, T. Matsui, "C^*-algebra net: a new approach generalizing neural network parameters to C^*-algebra", Proc. 39th Int. Conf. Machine Learning, PMLR 162 (2022), 8523–8534.